IIT M

End-to-End Automatic Speech Recognition



Automatic Speech Recognition

Automatic Speech Recognition (ASR)/ Speech to text (STT) is a technology that allows human to interact with devices through speech

Transforms audio into a sequence of words

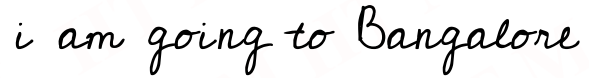
Applications:

- Information retrieval
- Assistive Technologies
- Data entry
- Dictation



Speech Recognition

Speech Recognition is easier to understand if you understand cursive handwriting recognition.



- The Basic units are "alphabets" of the written Language

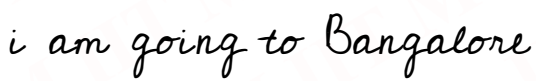
  26 (a... z) in English

  अ आ ... य र ल व श स ह (45 Hindi)

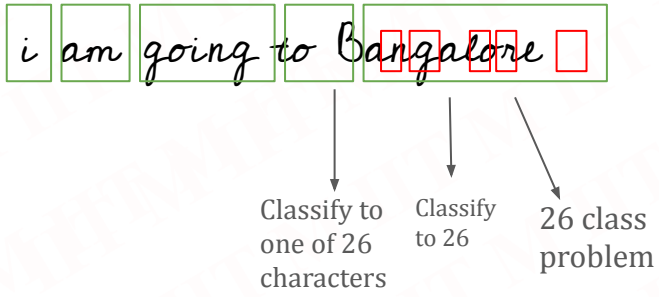
- The alphabets are concatenated to form words
- Words form sentences

Handwriting Recognition

- Converting handwritten form to editable text

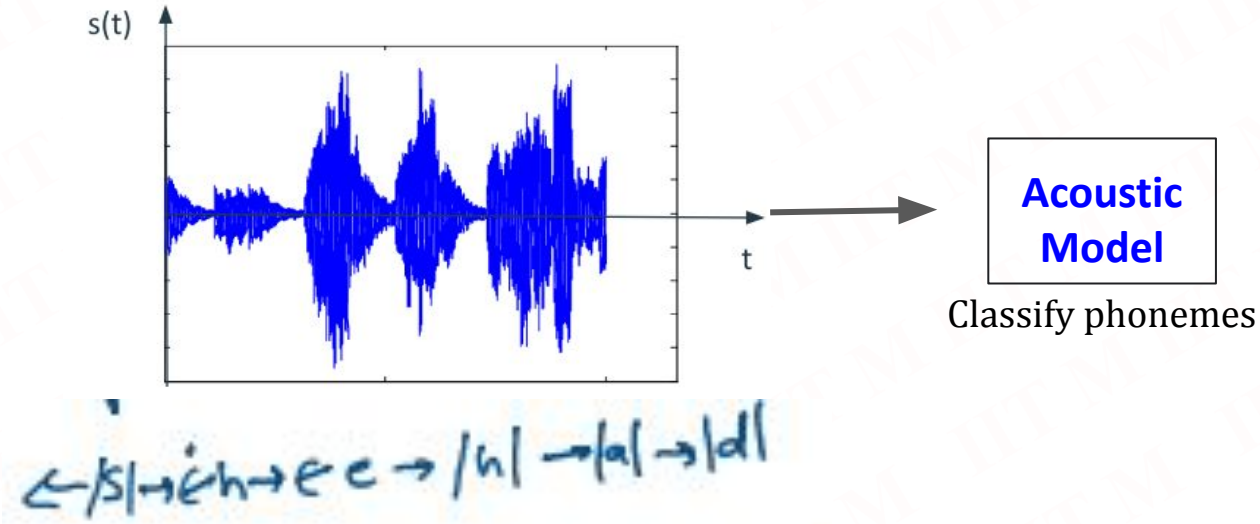


- Image Processing with segmentation → character and word level



- No clear boundaries between characters
- Words can be separated by "space"

Speech Recognition



- Spoken language is made up of 45-50 basic units - "phonemes"
- No distinct boundary between phonemes
- No distinct boundary between words

- Written Language
  - 26 alphabets (English)
  - 39 (Hindi)
- No distinct boundaries between characters
- Words usually separated by space

Review of Conventional ASR

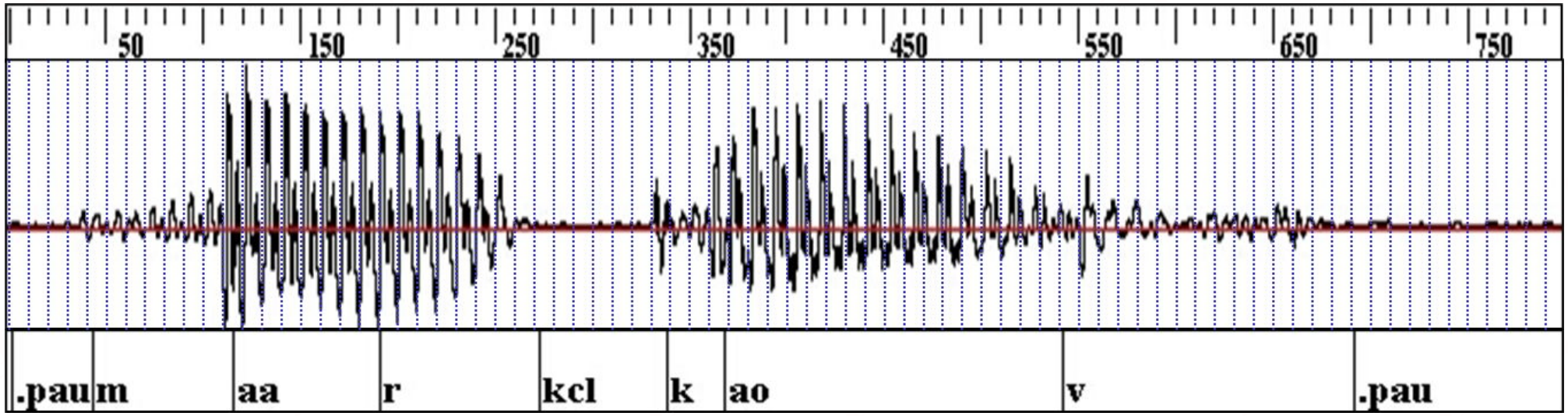
Acoustical Model: HMM-GMM to convert sequence of acoustic MFCC vectors to sequence of phonemes

Lexicon: Convert sequence of phonemes to words

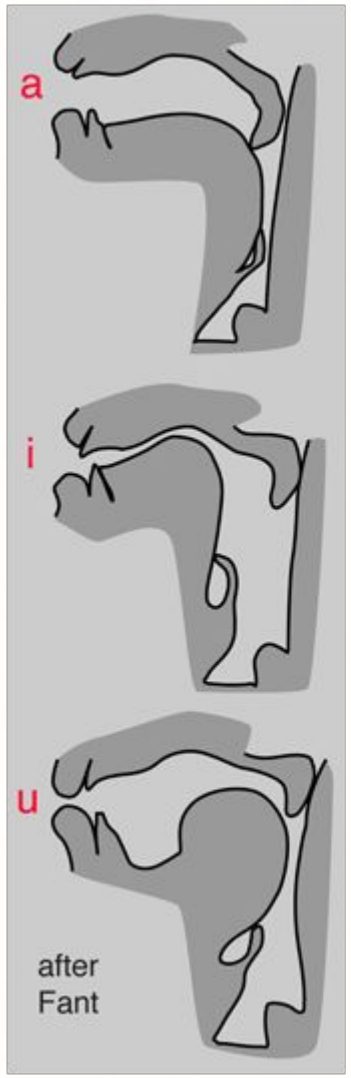
Language Model: To choose most probable sequence of words

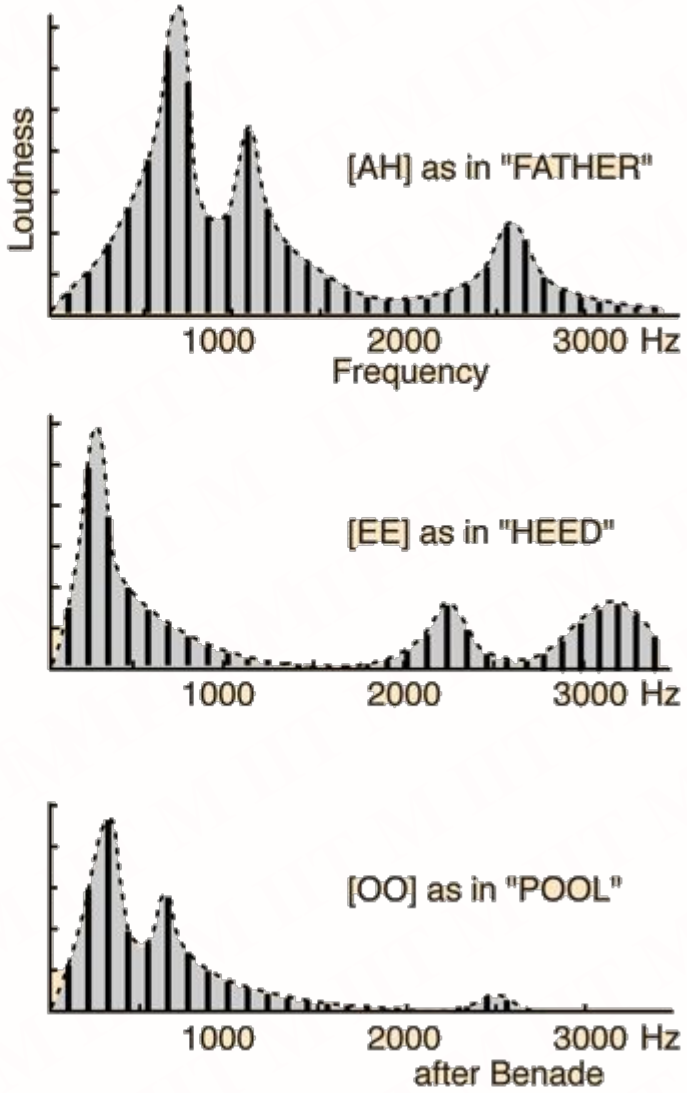
HMM For Speech Recognition

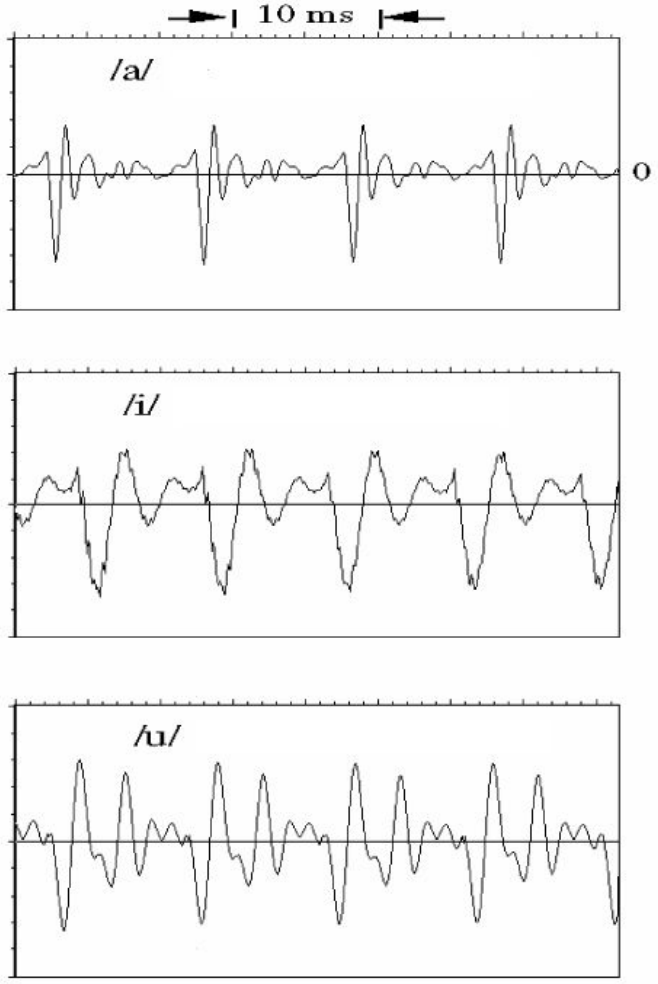
- Speech is divided into sequence of 10-msec frames, one frame per state transition (faster processing). Assume speech can be recognized using 10-msec chunks.
- Each vertical line corresponds to one observation
- Each state in a HMM can be associated with
  - sub-phoneme
  - phoneme
  - sub-word



Features: MOUTH CONFIGURATIONS & SOME VOWELS



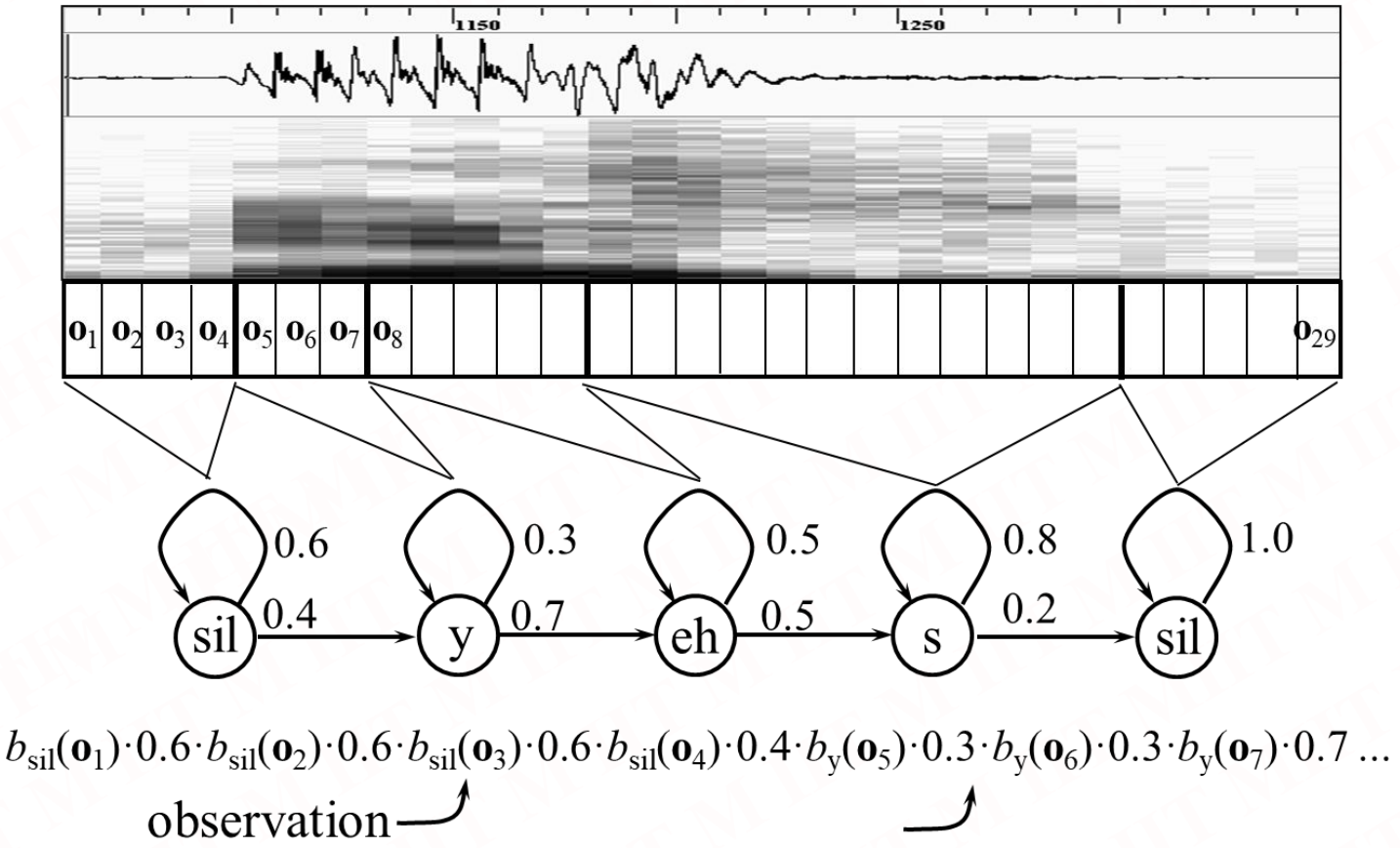




http://hyperphysics.phy-astr.gsu.edu/hbase/Music/vowel.html ,
http://research.spa.aalto.fi/publications/theses/lemmetty_mst/chap3.html

HMM For Speech Recognition

- Example of using HMM for word “yes” on an utterance:



Phonemes

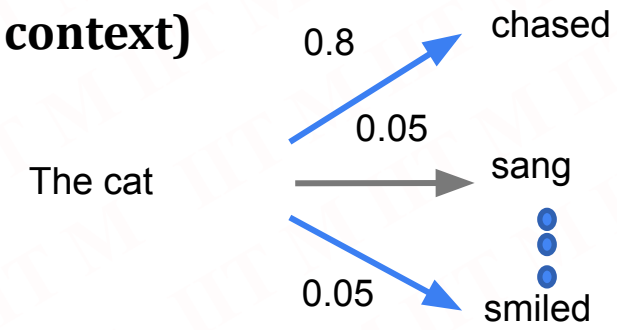
- Written language — Alphabets (a,x,s, அ, ஆ, ஐ, ர)
- Spoken language — Phonemes (ih, ay, aa)
- Lexicon is a link between acoustic-level representation and the word sequence output by the speech recognizer.
- For example,

ih t s f ah n t uw r eh k ah g n ay z s p iy ch

example	ih g z ae m p ah l
idea	ay d iy ah
simply	s ih m p l iy
possible	p aa s ah b ah l

Use Language Model to improve performance and overcome “phoneme errors”

P(current word|context)



ih t s f ah n t uw r eh k ah g n ay z s p iy ch

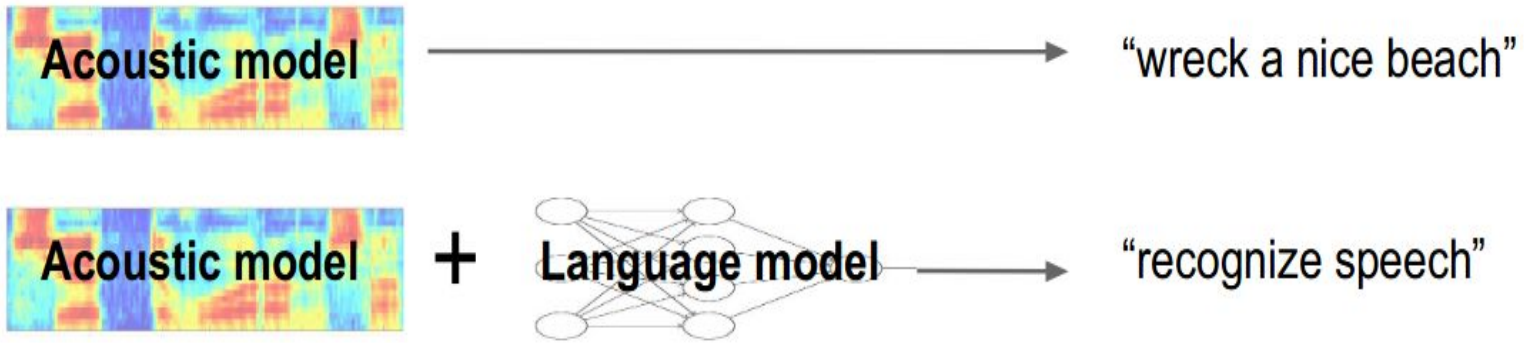
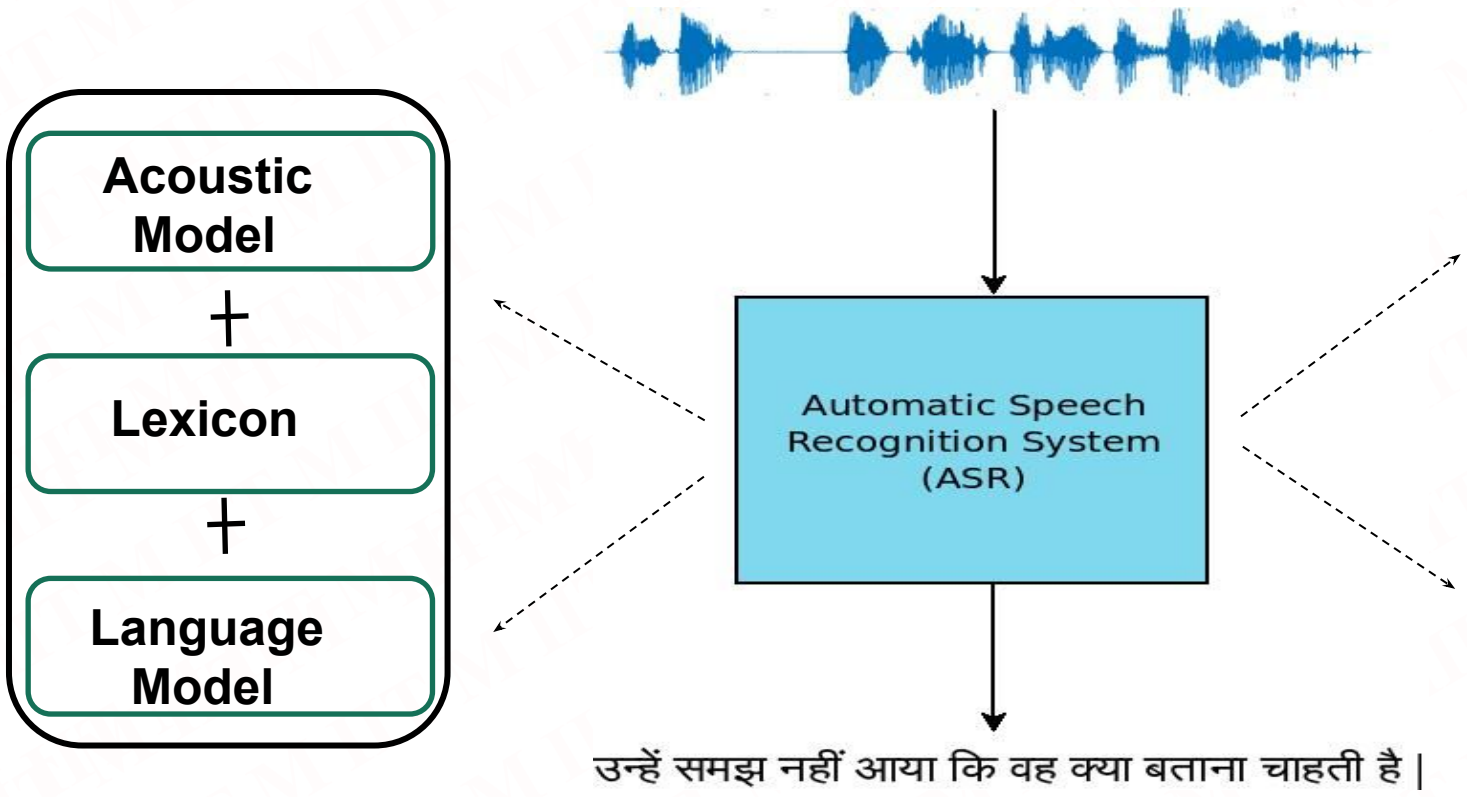


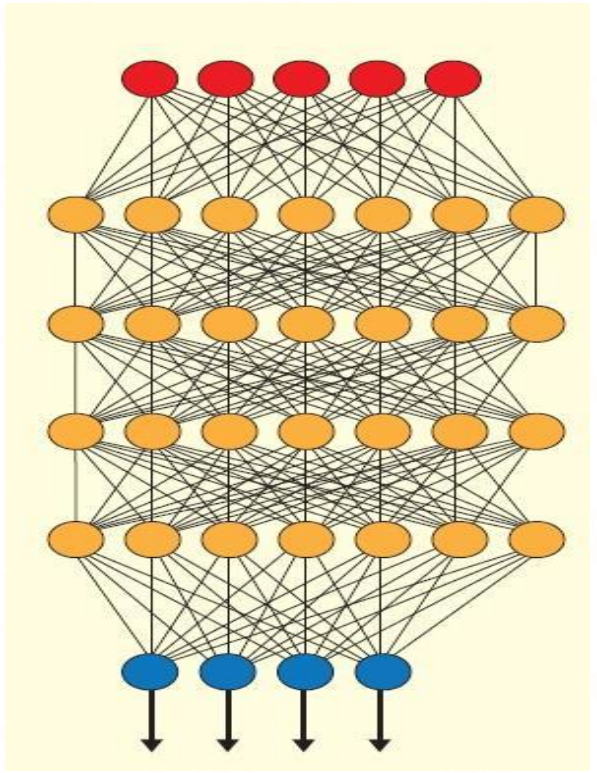
Image Source: https://mycourses.aalto.fi/pluginfile.php/398127/mod_resource/content/1/NNLM.pdf

Deep Learning Based ASR

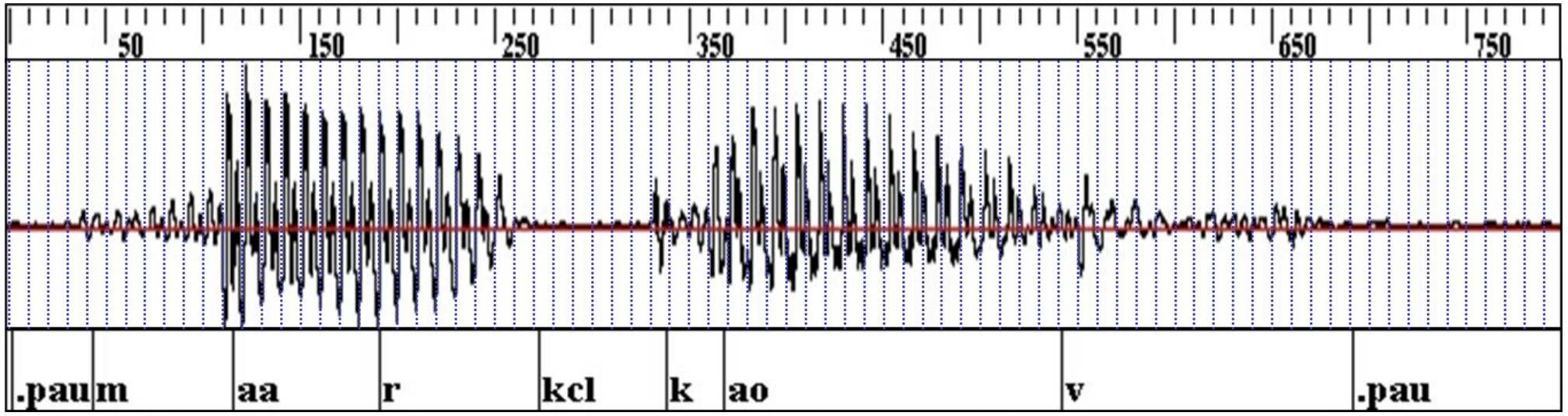
Conventional ASR System



End-to-End ASR System



End-to-End Speech Recognition

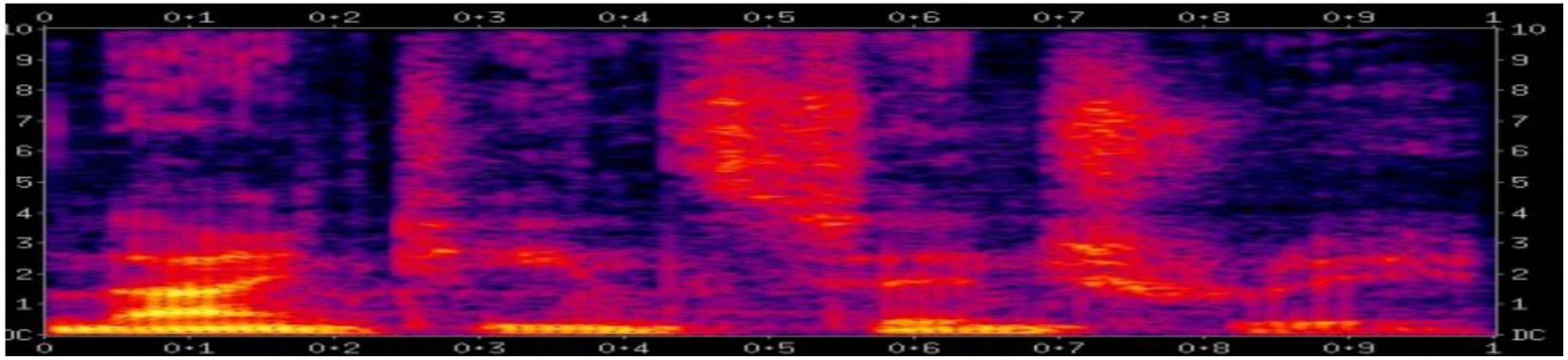


- Recall: In Conventional ASR we mapped speech acoustic vectors to “phonemes”
- In End-to-End ASR
  - Given speech acoustic vectors, we will try to directly get the outputs as “words”
  - Or output as characters which can repeat and may have “blanks” in between
    - We remove the duplicates and blanks from the outputs to get words

Modelling Sequences

In many real world scenarios, we have to consider sequences of inputs to produce an output. For example,

Speech recognition:



Text analysis:

- Analyze document, identify topic
- Input English, Output French

Stock Prediction:

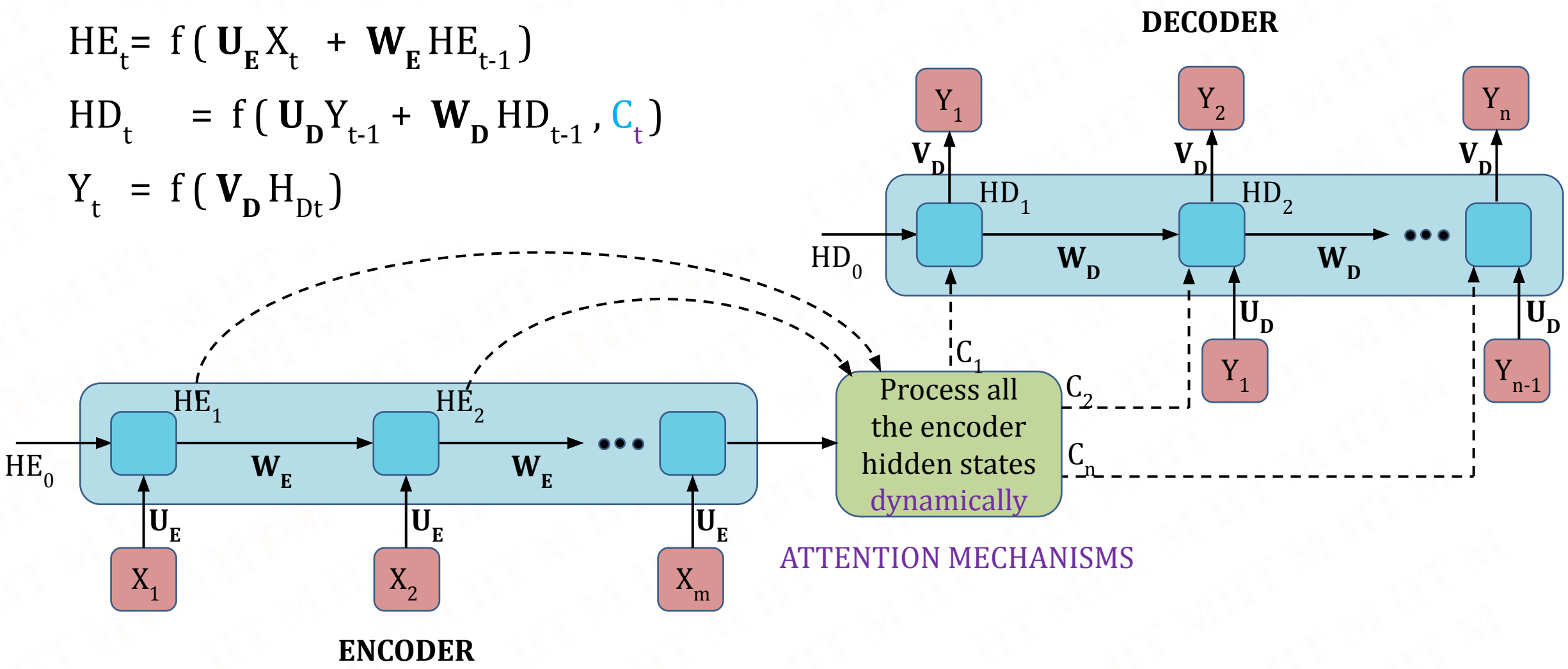
- Stock analysis using previous week's data

Sequence to Sequence Models (SEQ2SEQ)

$$HE_t = f(\mathbf{U_E} X_t + \mathbf{W_E} HE_{t-1})$$

$$HD_t = f(\mathbf{U_D} Y_{t-1} + \mathbf{W_D} HD_{t-1}, C_t)$$

$$Y_t = f(\mathbf{V_D} H_{Dt})$$



Transformer

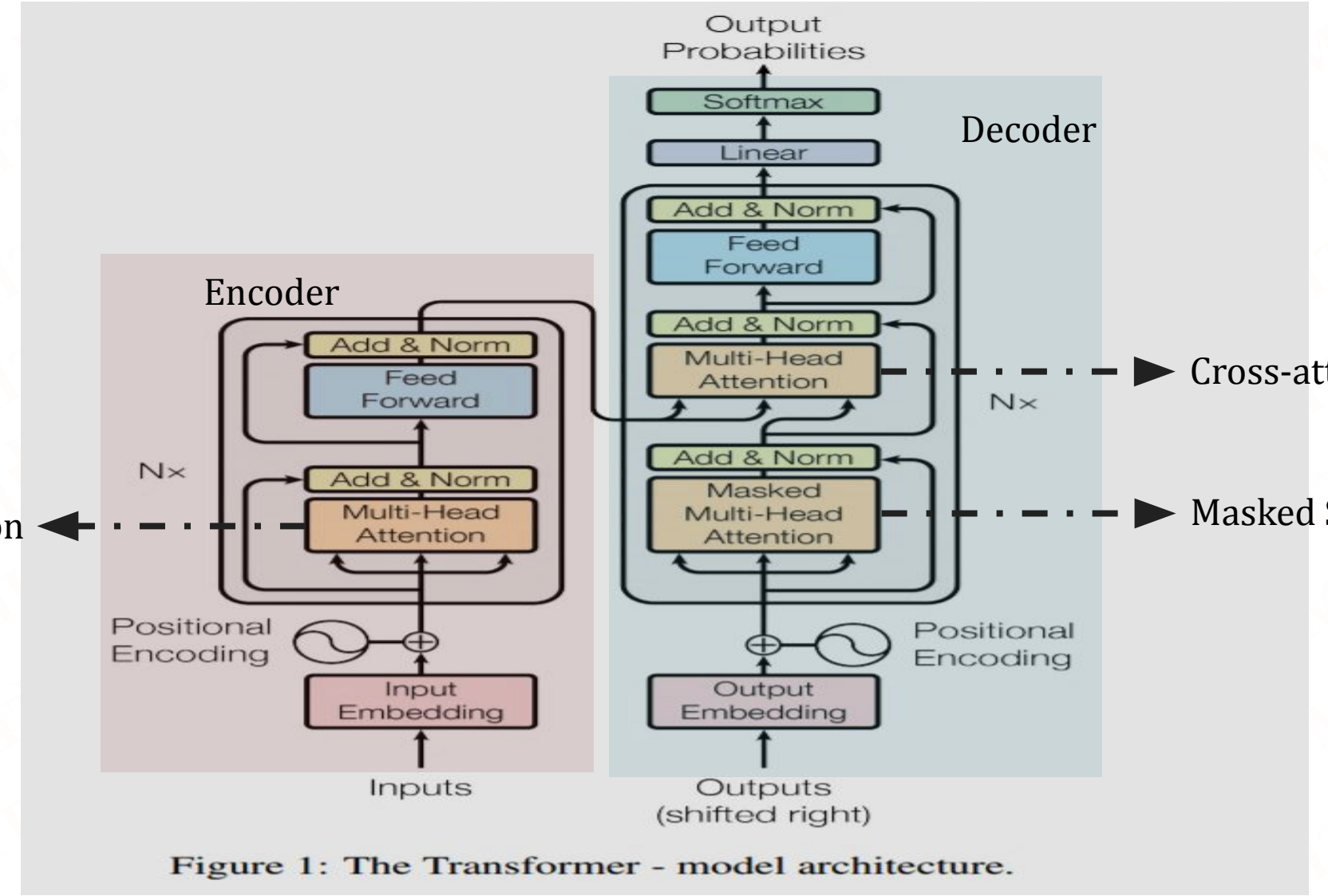
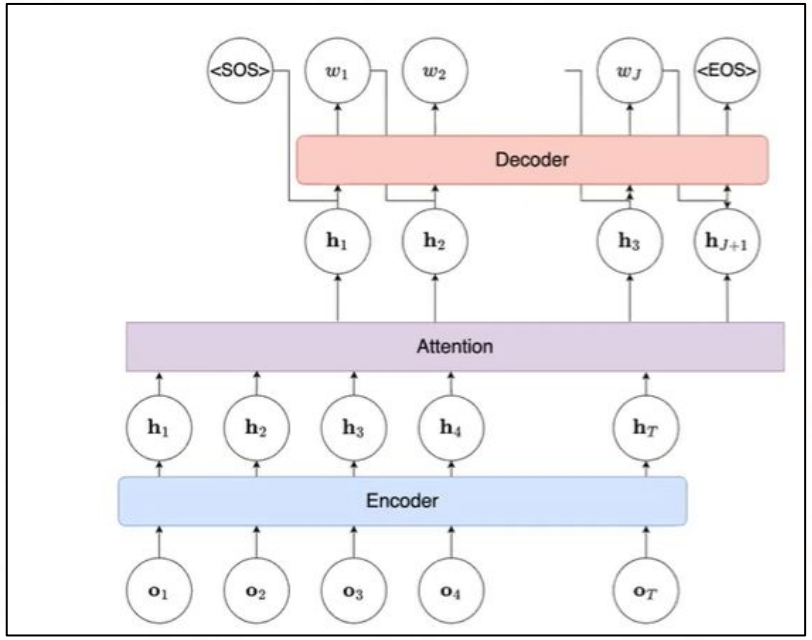


Figure 1: The Transformer - model architecture.

End-to-End ASR Models built on Encoder-Decoder Architecture

Portions of the material are borrowed and adapted from external video lectures, such as those from WAVLAB.

Listen-Attend-Spell-Encoder-decoder Google

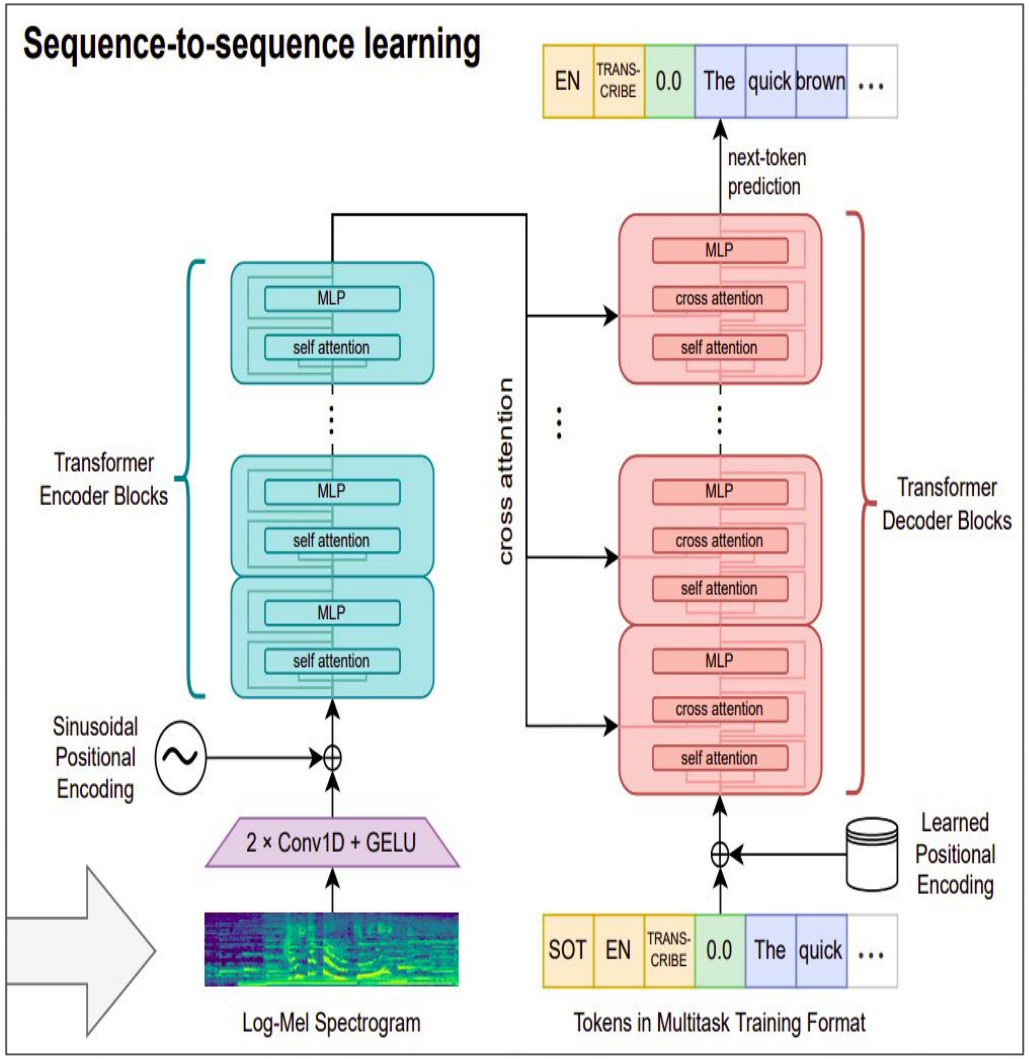


Beginning of attention-based ASR

- Encoder: CNN (down sampling) + BLSTM x N layers
- Decoder : LSTM x M layers
- Attention: Convolution based neural-network attention

- Chorowski, Jan K., et al. "Attention-based models for speech recognition." Advances in neural information processing systems 28 (2015).
- Chan, William, et al. "Listen, attend and spell: A neural network for large vocabulary conversational speech recognition." 2016 IEEE international conference on acoustics, speech and signal processing (ICASSP). IEEE, 2016.
- Hori, Takaaki, et al. "Advances in Joint CTC-Attention based End-to-End Speech Recognition with a Deep CNN Encoder and RNN-LM." (2017).

Sequence-to-Sequence learning

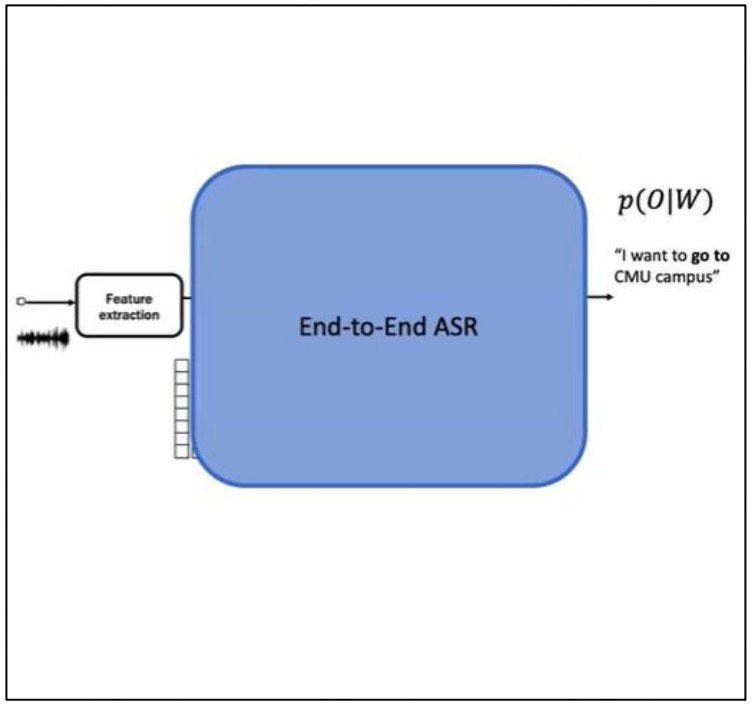


What units to use as output for E2E ASR?

End-to-End ASR case

How to describe the phrase “go to” ?

- **Word**
  - “Go” and “to”
  - More **semantic/syntactic**
  - Very large vocabulary size, e.g., $|V|$ would be 100k
  - **Out of Vocabulary issue**
  - The length (*N=2)* is very short. **Less** Computational cost, but **larger** mismatch between the input and output lengths
- **Character**
  - "g" "o” "_" ”t" and "o" ("_" means the space)
  - The vocabulary size is not large in general. ~30 in the Roman cript, ~10K in the Chinese script
  - No out of vocabulary issue (rarely happens)
  - The length (N = 5) becomes longer. More computational cost but relaxing the mismatch between the input and output lengths.



- **BPE: Byte Pair Encoding (sentence piece)**
  - "go to" → "-8" "o" "_to" (N = 3)
  - Something between, we can also control the vocabulary size

In general, we do not specify which unit we use in our lecture since this is one of the model configurations.

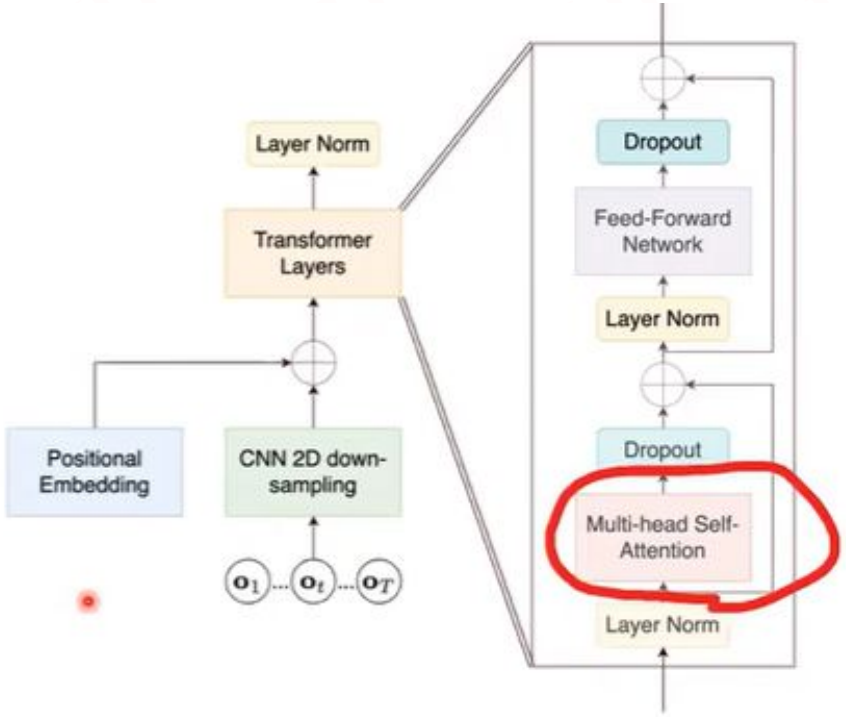
The Encoder-Decoder ASR Problem Formulation

Attention based encoder-decoder

- Let $W = (w; \epsilon V | j = 1,..,J)$, be a token sequence
- Let $O = (\mathbf{o}_t \epsilon \mathbb{R}^D | t = 1, ..., T)$, be a sequence of D dimensional
- feature vectors
    - $\hat{W} = argmax_w p(W|O)$
- Problem: T and J are different, and we cannot use normal neural
- networks
- Sequence to sequence is a solution to deal with it

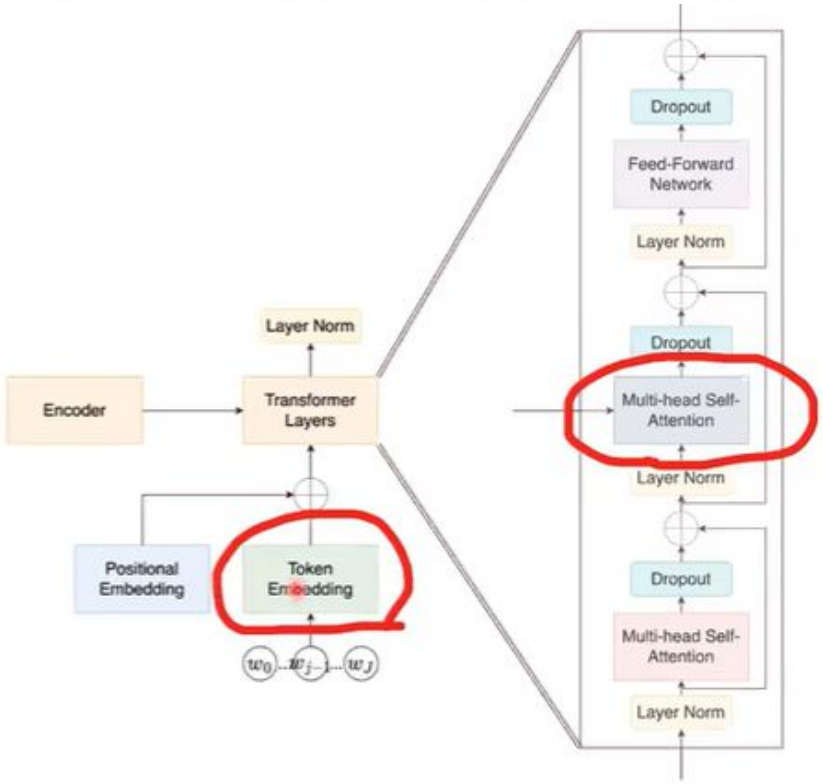
ASR Transformer Encoder

- ***Multi-head*** extension
  - $\mathbf{h}_{j,h} = \sum^{T}_{t=1} a_{jt,h}\, v_{t,h}$ *h*: index of head
  - All query,key,value matrix depends on the head, e.g., $(\mathbf{w}_{q,}\mathbf{b}_{q}) \rightarrow (\mathbf{w}^{q}_{h,}\mathbf{b}^{q}_{h})$
  - $\mathbf{h}_{j} = \text{Concat}(\mathbf{h}_{j,1},\mathbf{h}_{j,2},....\mathbf{h}_{j,H})$: *H* is the number of heads



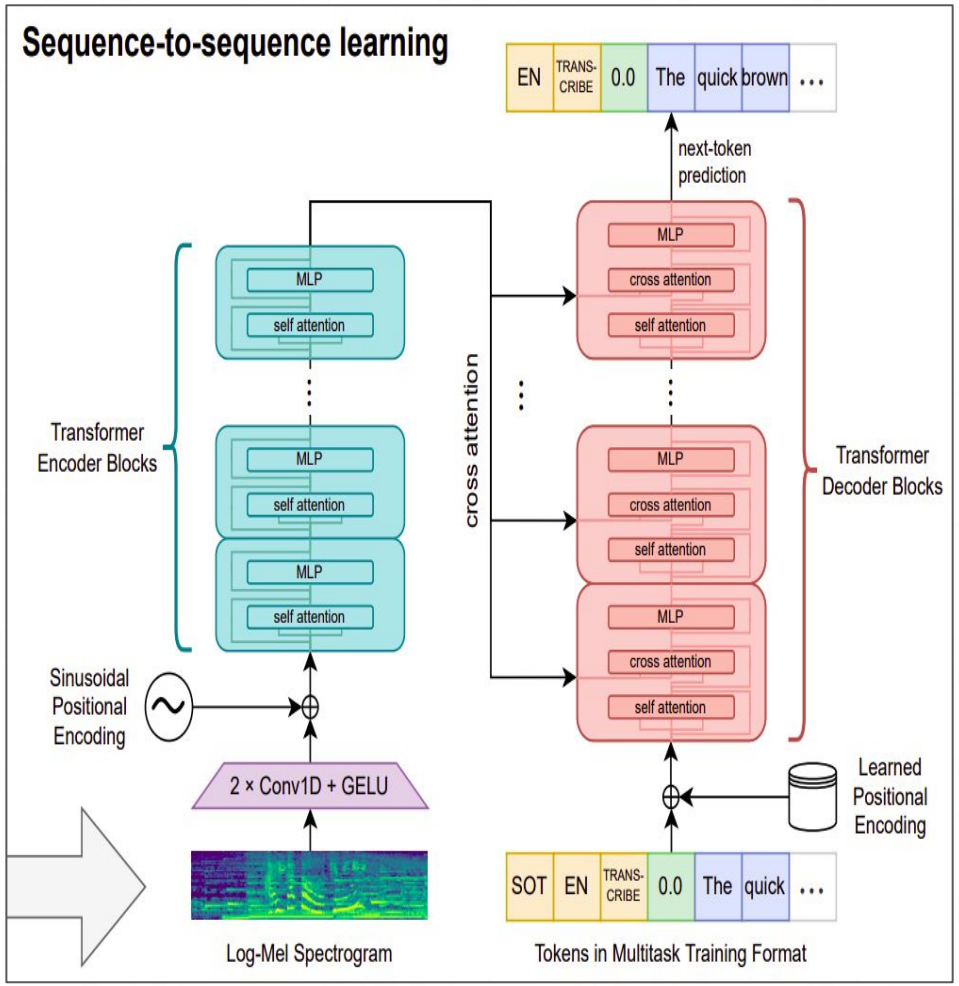
ASR Transformer Decoder

- ***Same as the transformer encoder except for***
  - CNN downsampling Token embedding
  - Input : Previous token conditioned on the encoder vector ($\mathbf{h}_1, \mathbf{h}_2, \ldots \mathbf{h}_t \ldots$),
  - Insert the cross attention between the multi-head extension and the feed-forward network



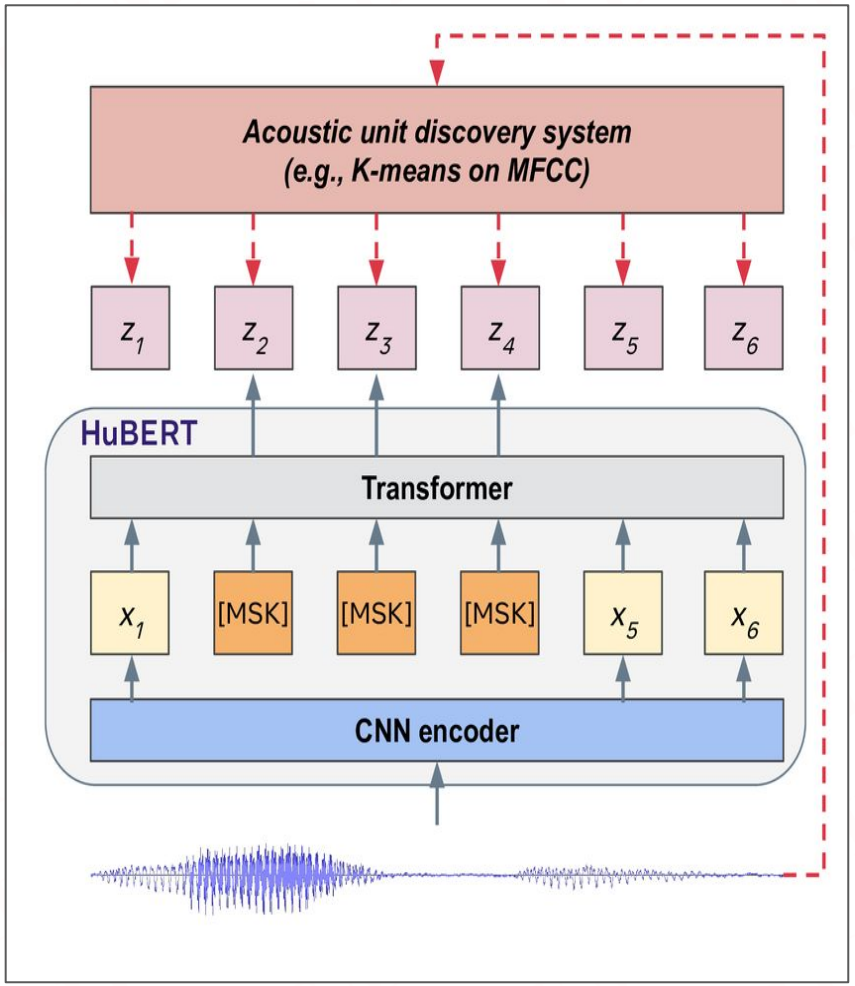
Three Broad Architectures for ASR based on Transformers

Encoder-Decoder
Eg: OpenAI Whisper



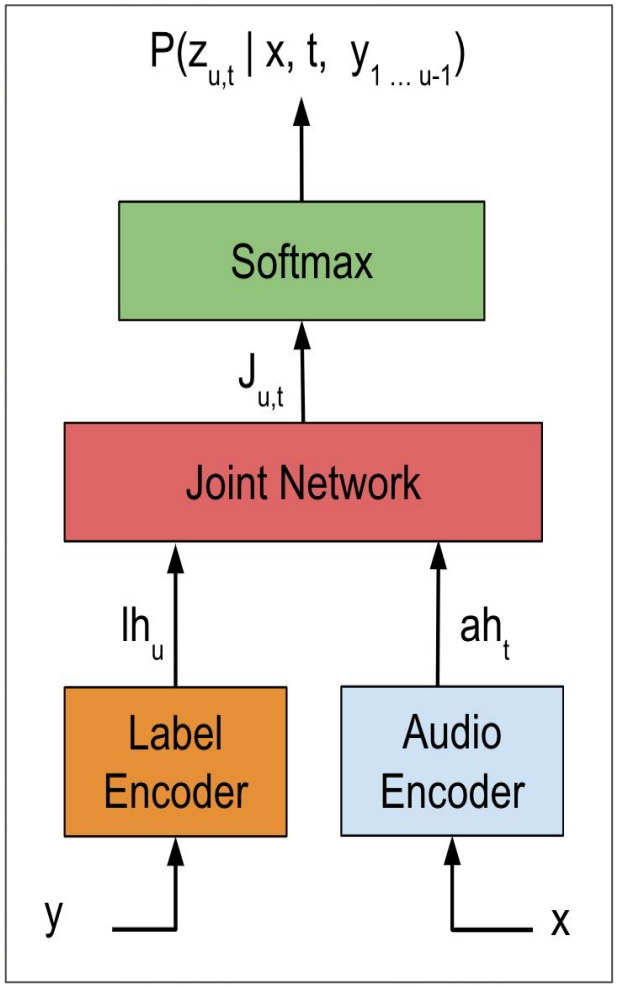
- Disadvantage: Decoder is autoregressive but acts like an audio-conditioned language model
- Cannot be easily used for streaming

Encoder only with CTC loss
Eg: HUBERT



- Non-Autoregressive and hence faster
- Does not use an explicit "Language Model"

Transducer
Eg: K2 Zipformer

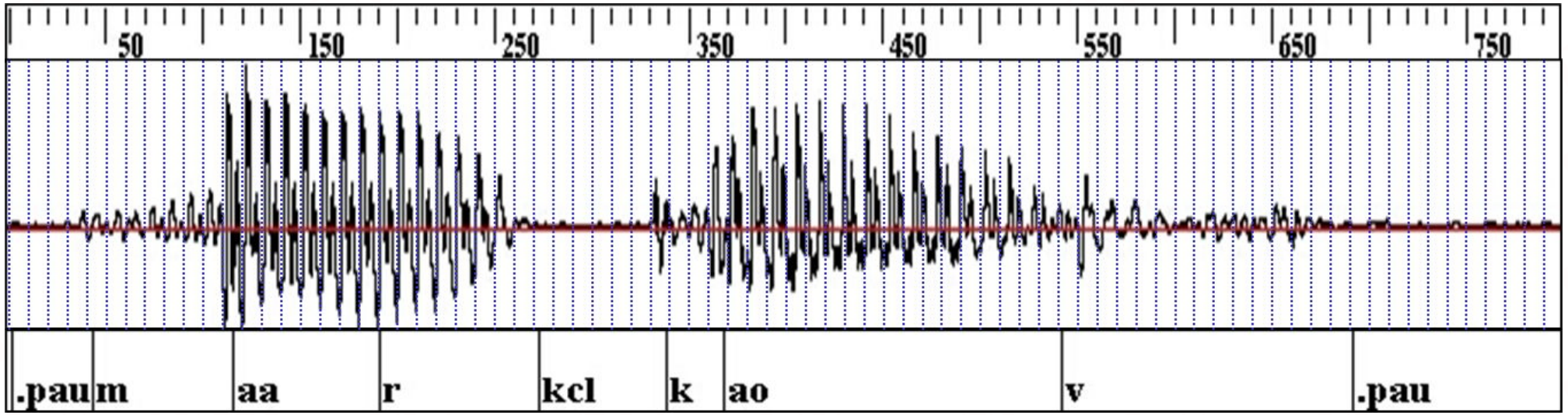


- Good for streaming
- Used by most companies
- Has the power of language model

Alignments

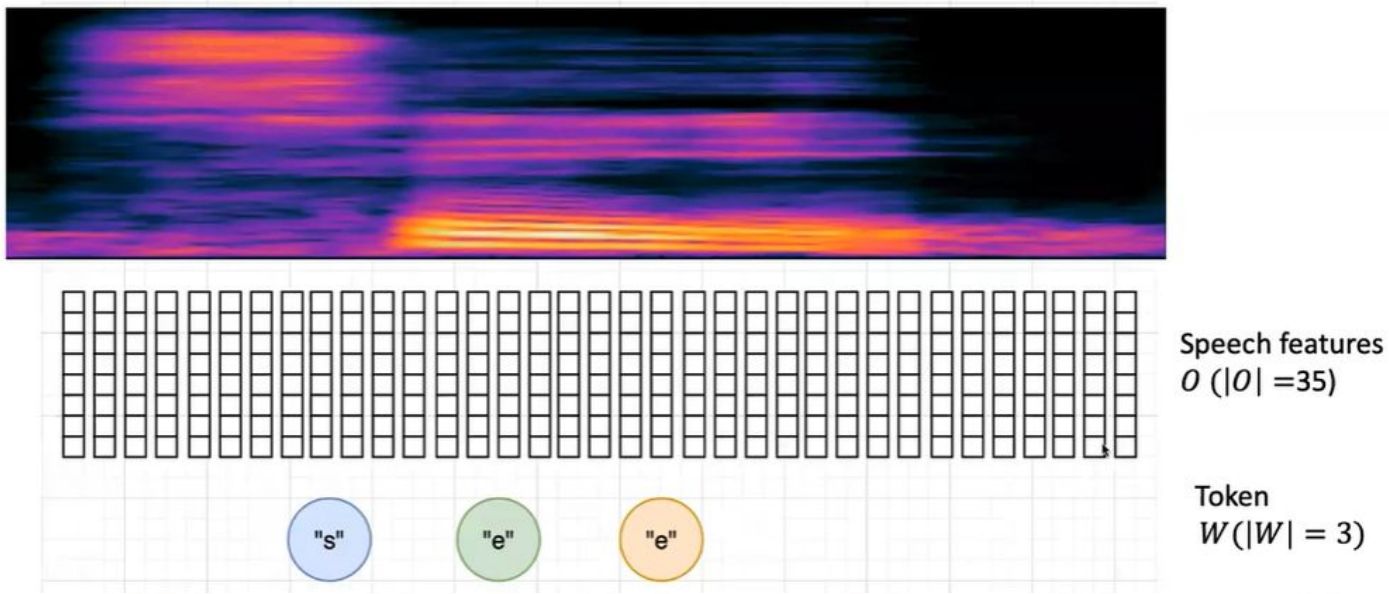
- Recall in HMM-GMMs
  - You needed an alignment between “frame” (i.e. acoustic vector) and “state” (i.e. phone)
  - Hard Alignment: Viterbi

End-to-End Speech Recognition



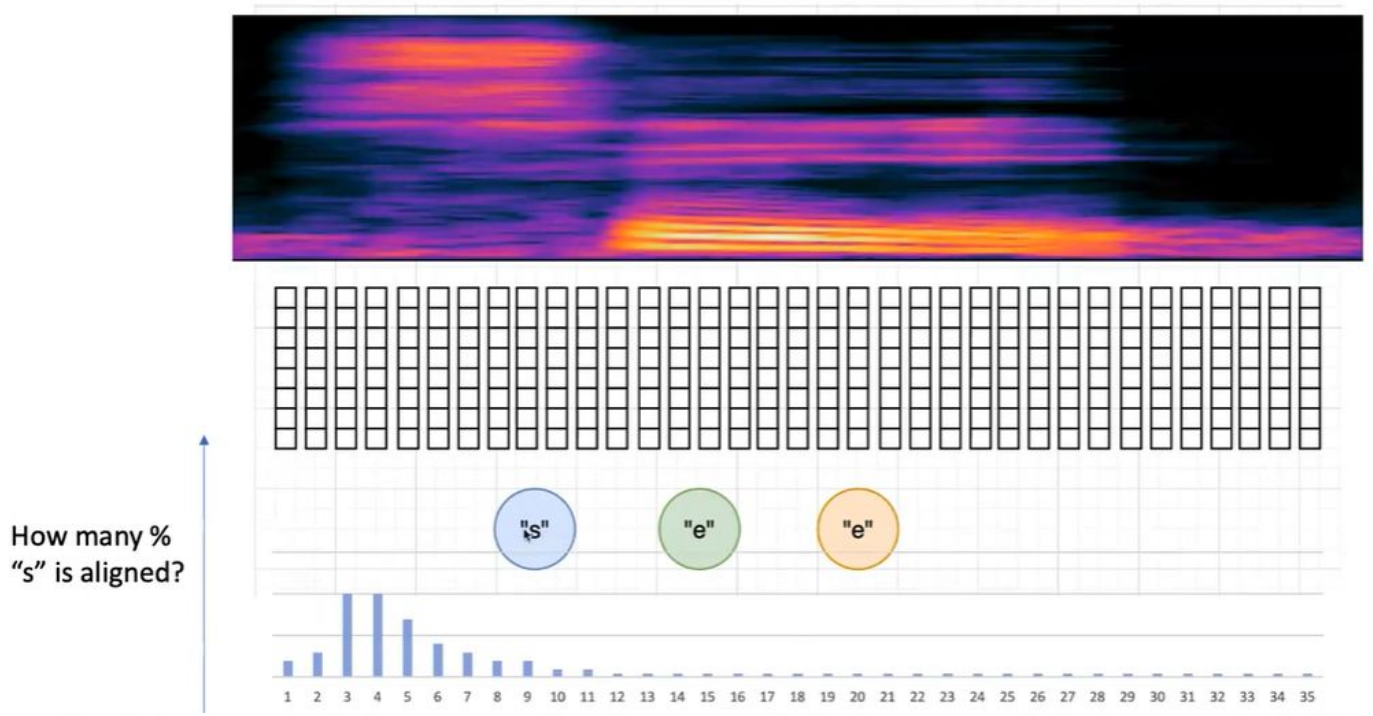
- Viterbi will give “acoustic frame” to phoneme (state) alignment

Alignments in End-to-End between text and acoustic vectors



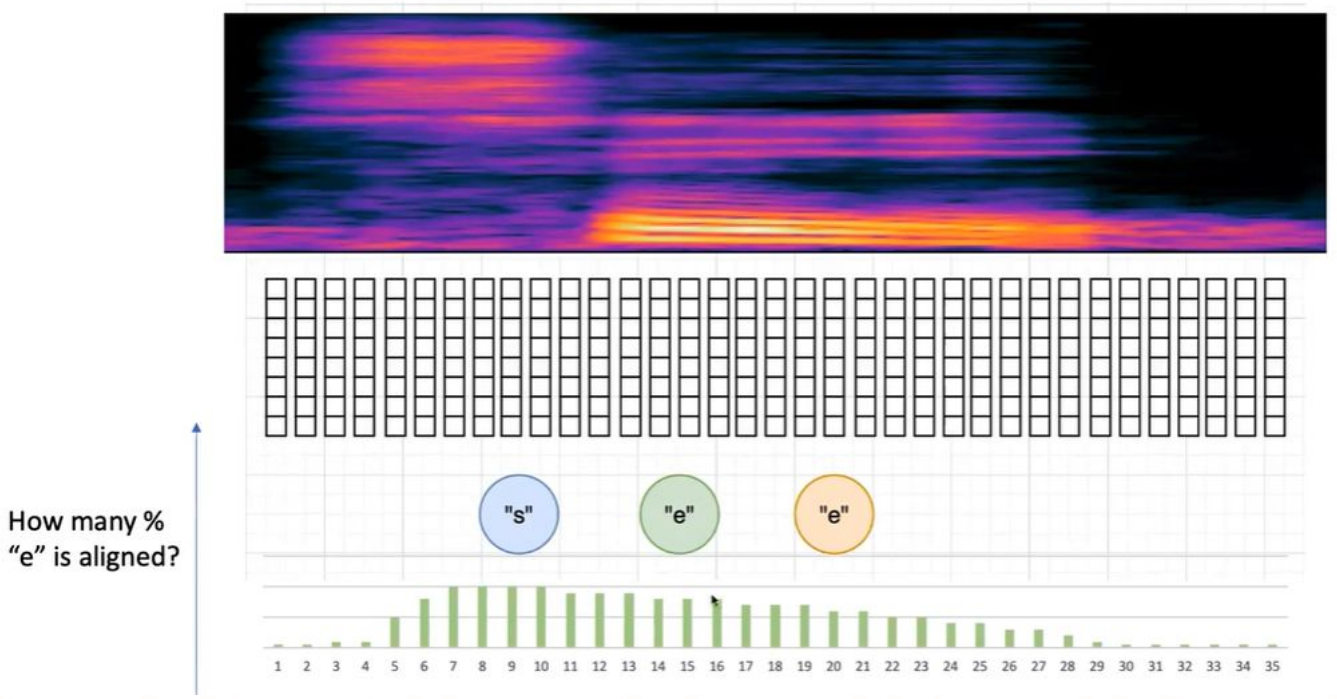
Portions of the material are borrowed and adapted from external video lectures, such as those from WAVLAB,CMU.

Attention is Soft Alignment



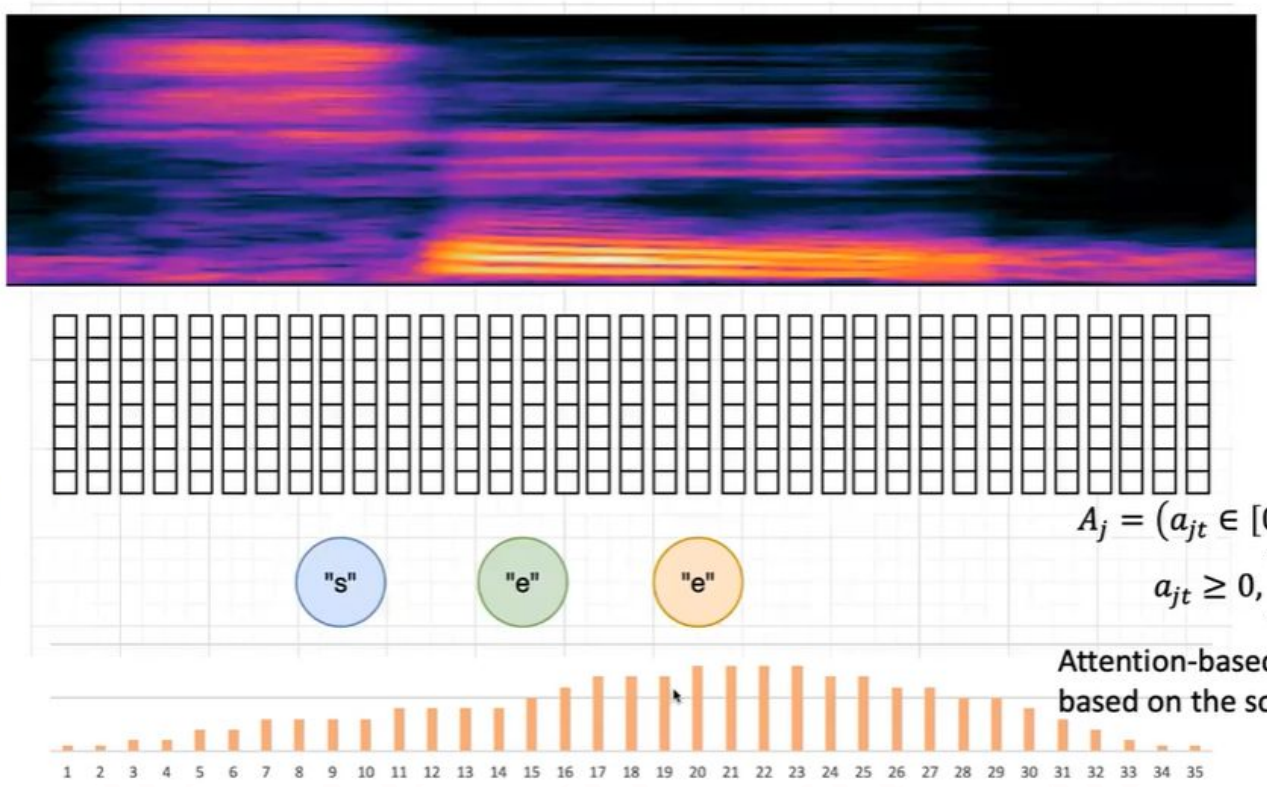
Portions of the material are borrowed and adapted from external video lectures, such as those from WAVLAB,CMU.

Soft-Alignment for Character “ green e”



Portions of the material are borrowed and adapted from external video lectures, such as those from WAVLAB,CMU.

Soft alignment with “orange e”



$$A_j = \left(a_{jt} \in [0,1] \middle| t = 1, \ldots, T\right)$$

$$a_{jt} \geq 0, \sum_t a_{jt} = 1$$

Attention-based approach is based on the soft alignment

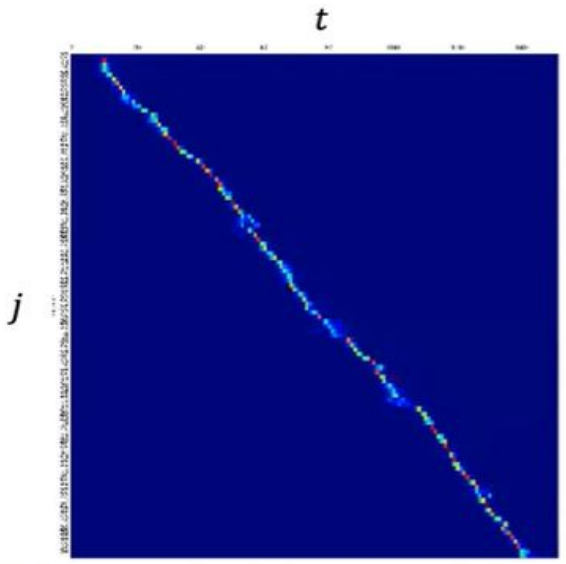
Monotonic nature of alignment

- As we go from left to right
  - The “left” characters have more probability with “left frames”
  - The “right characters” have more probability with “right frames”

- This is VERY DIFFERENT FROM TEXT-TRANSLATION
  - Where word orders can change between the languages in the translation

- However, note “s” has NON-ZERO probability at right-most frame
  - So “too much” freedom!

Cross-Attention in Encoder Decoder



- $\mathbf{h}_j = \sum_{t=1}^{T} a_{jt}\, \mathbf{v}_t$
- Attention weight a_{jt} determines whether encoder $\mathbf{v}_t$ is assigned to a token w_j or not
  - $a_{jt} \approx 0$: no assignment
  - $a_{jt} \neq 0$: assigned

Attention Problem Formulation

$$p(W|O) = \prod_j p(w_j|w_{1:j-1}, \mathbf{h}_j)$$

First, we transform encoder vector h_t to the hidden state v_t

$$\mathbf{v}_t = \mathbf{W}^{\mathbf{v}}\mathbf{h}_t + \mathbf{b}^{\mathbf{v}}$$ (called value)

Obtain the context vector for the decoder cross-attention

$$\mathbf{h}_j = \sum_{t=1}^{T} a_{jt}\,\mathbf{v}_t$$

Compute the assignment probability for each output *j* from a neural network

$$\mathbf{a}_j = \{a_{jt} | t = 1, \ldots, T\} \in \mathbb{R}^T,\ 0 < a_{jt} < 1, \sum_{t=1}^{T} a_{jt} = 1$$

a_{jt} is obtained by using a neural network

Cross-Attention and encoder embeddings

- Desired property of $\mathbf{h}_j$
  - Desired property of $\mathbf{h}_j$
  - Has some information corresponding to w_j
  - Has some associations (alignment) with speech input $\mathbf{O}$
  - Differentiable
  - Let's use the **attention mechanism**!

$$p(W|O) = \prod_{j=1}^{J} p(w_j|w_{1:j-1}, O)$$

$$\approx \prod_{j=1}^{J} p(w_j|w_{1:j-1}, \mathbf{h}_j = \text{Attention}(O))$$

Extension -- Conformer

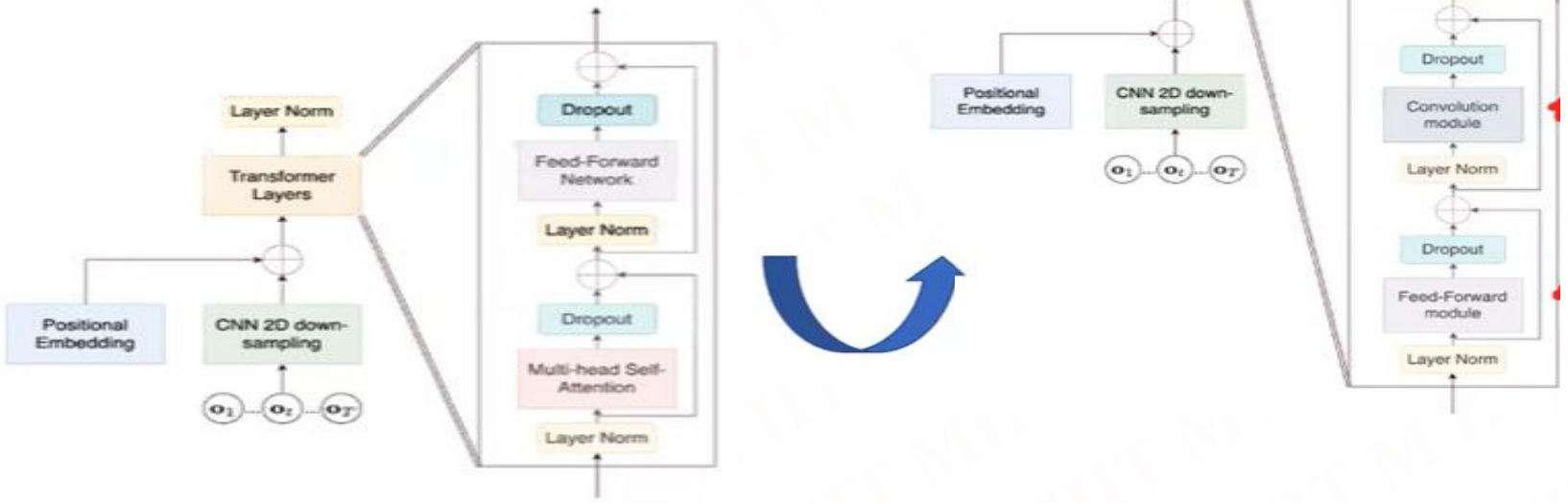
- Convolution-augmented Transformer for Speech Recognition
- **Global** information: captured by a self-attention network
- **Local** information (important for speech!): captured by a convolutional neural network
- It is originally proposed by an RNN-Transducer architecture
    - The conformer encoder is widely used for the other architecture, including attention-based ASR and CTC

Gulati, Anmol, et al. “Conformer: Convolution-augmented Transformer for Speech Recognition." *Proc. Interspeech 2020* (2020): 5036-5040.

Transformer to Conformer

From transformer to conformer

- Two feed-forward layers (Macron)
- Convolution block
- Absolute -> relative positional embedding



LM is not very useful for Attention-Encoder-Decoder

- Attention-based ASR

$$p(W|O) = \prod_j p(w_j|w_{1:j-1}, O)$$

  - No language model term compared with HMM-based ASR $p(W|O) = p(O|W)\,p(W)/p(O)$
  - Language model $p(W)$ is built only with the text data W (book,web data,etc.,)

- Shallow fusion

$$p(W|O)p^{\alpha}(W) = \prod_j p(w_j|w_{1:j-1}, O)p^{\alpha}(w_j|w_{1:j-1},)$$

α : language model weight

- This is not the perfect solution. We duplicate the language model effects
  - One for $p(W)$ and the other for the decoder network in $p(W|O)$
  - Residual LM subtraction techniques have been developed

Attention-Based ASR

- No conditional independence assumption
- Cross-attention for the alignment
- Transformer improved the performance significantly

- Attention is too flexible → Hard alignment approaches based on CTC and RNN-T

- Autoregressive.. So is slow as it generates “token-by-token”
- In its simplest implementation – it has to first get entire speech-encoding before cross-attention (real-time ??) and decoding

- Can we “parallise” or make it “non-autoregressive”?
  - CTC is one approach for “non-autoregressive”

CTC ASR Problem Statement

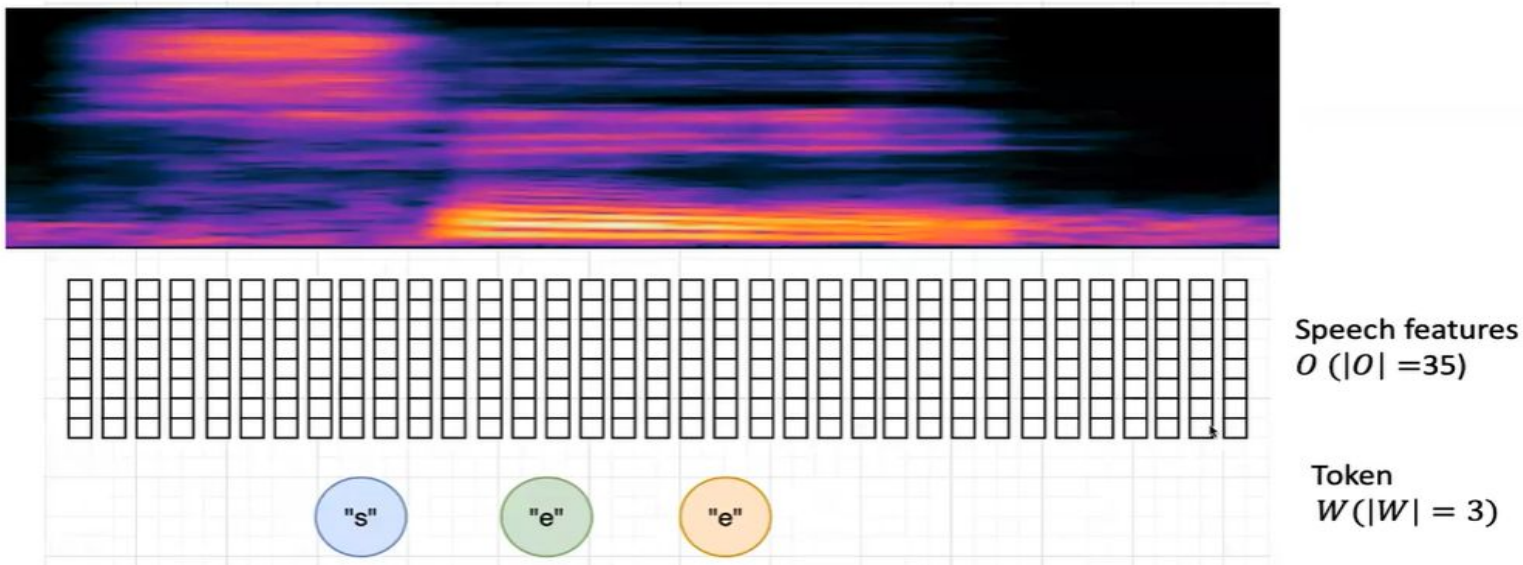
- Again, our starting point *p(W|0)*
- We introduce a hard alignment path as a random variable: Z = *(z+ ϵ V'|t = 1, .., T)*
- We consider the following equations

$$p(W|O) = \sum_{Z} p(Z, W|O)$$

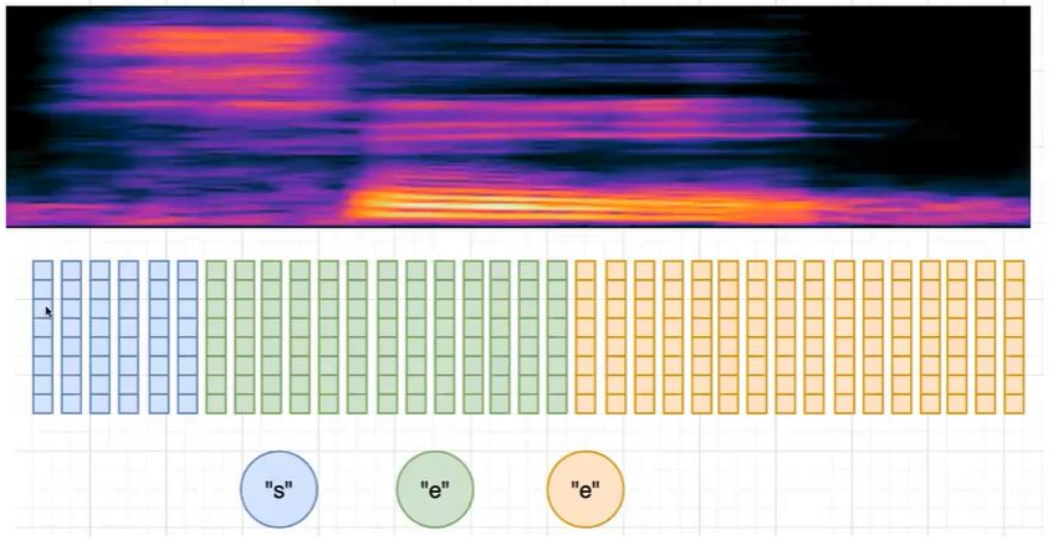
$$= \sum_{Z} p(Z|O)p(W|Z, O)$$

Recall: {\bf O} is usually like 100 frames per sec, while W the text is about 10-12 chars/sec

Recall: Alignments

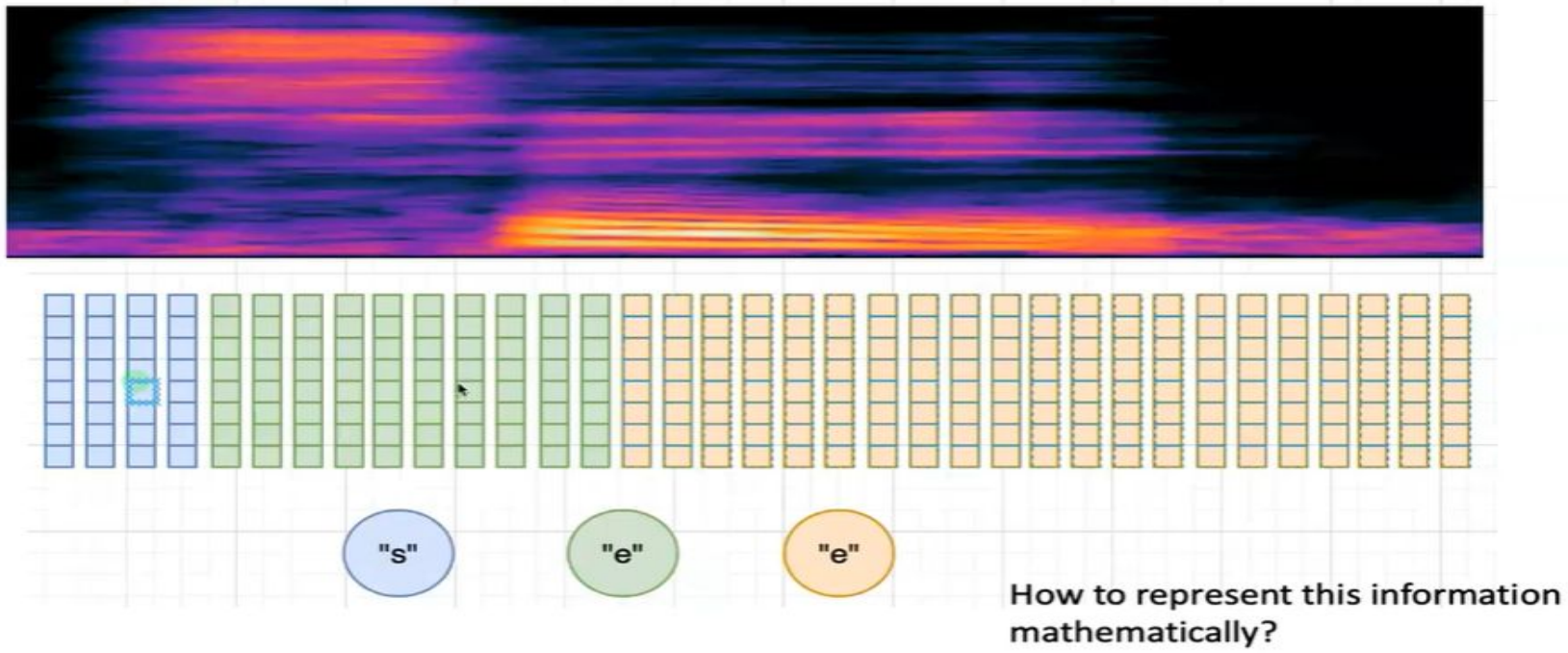


Hard Alignments – Example1



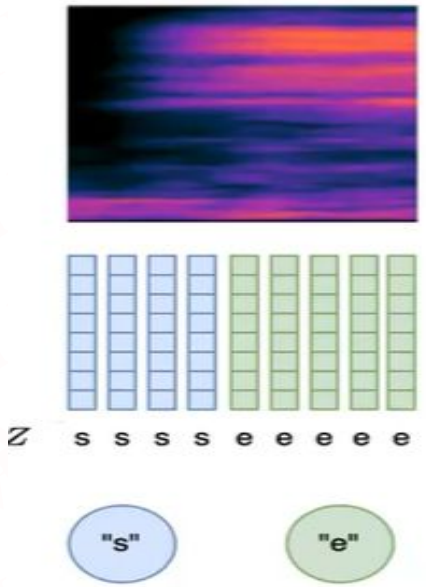
- This is an example of a VALID HARD-ALIGNMENT between characters and frames
- “Hard” means the frame to exactly one character. Recall in “soft” every frame had non-zero probability for all characters
- Also there is “strong monotonicity”. After frames are assigned to “s” only can we assign frames to “e”..

Hard Alignments – Example2



- This is another VALID HARD ALIGNMENT for the SAME WORD and FRAMES

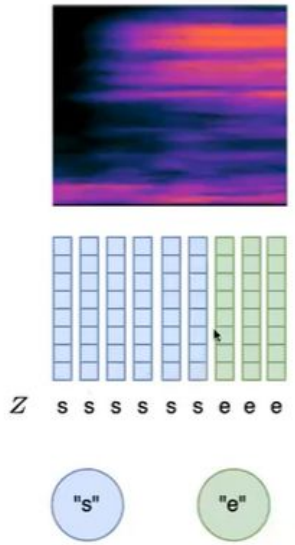
Introduce “alignment variable” to represent Hard Alignment



- z_1 =“s”, z_2 =“s”, .., z_5 =“e”, z_6 =“e”,

- We introduce an alignment variable “z” that indicates which frame is associated with which character
- There will be AS MANY Z VARIABLES as THERE ARE NUMBER OF FRAMES
- Recall Hidden Variable “z” in GMM which shows which observation is associate with which Gaussian component

Alignment Variables change when our Alignment Change



- z_1 ="s", z_2 ="s", .., z_7 ="e", z_8 ="e",

Connectionist Temporal Classification – Alex Graves & Schmidhuber

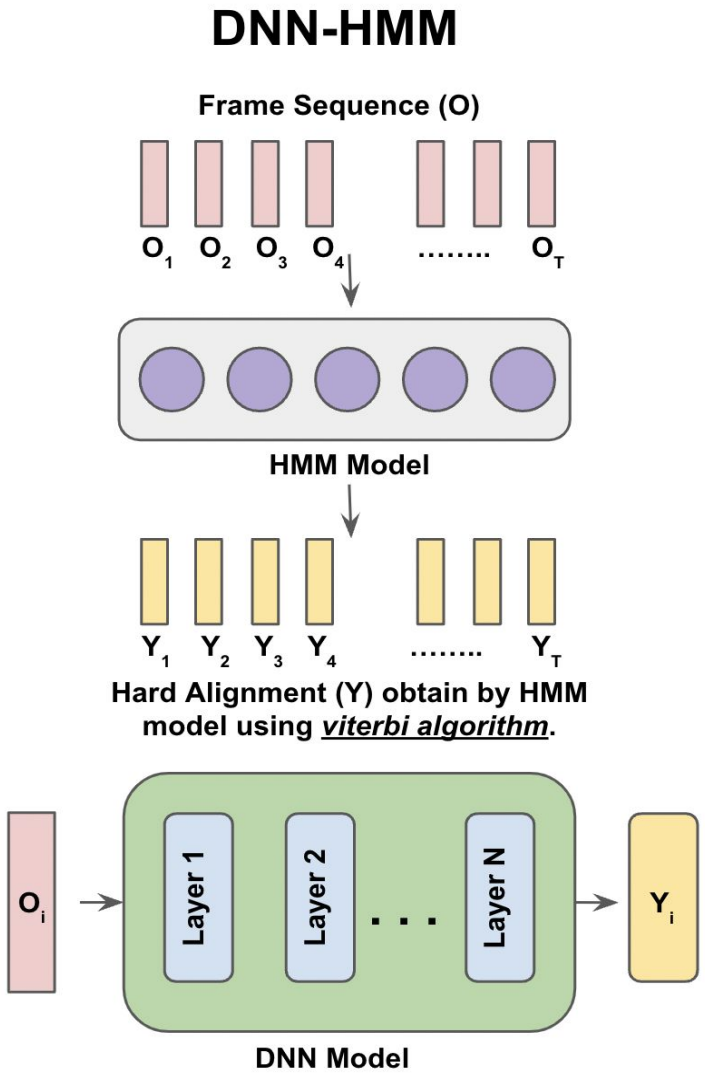
- Basic idea is: we should try to maximise the correct word output given the observation sequence
- Not have to worry about alignments – which can have many "reasonable" assignments

- This was important in 2005-2006
  - Graves/Schmidhuber proposed using RNNs for ASR
    - This was during the time when HMM-GMMs were powerful
  - Training was complicated since
    - At each RNN step they had to output a class assignment

CTC based End-2-End ASR

CTC : Connectionist Temporal Classification

Conventional Approach (HMM-DNN)



- Train an HMM based acoustic model
- Obtain Alignments (**frame-phoneme map**) from the HMM based model (using viterbi algorithm).
- Use **this alignment as the ground truth** to train a Neural Network (replaces GMM):
  - DNN (Deep Neural Network), TDNN (Time Delayed Neural Networks), etc.

Problem with conventional method

- The major problem with the conventional method is the **dependency of frame to phoneme alignments obtained by HMM models to train DNN models.**

- What if we do not have such alignments ?

- Instead, we only have **input sequence of acoustic frames and the corresponding sequence of output tokens (words).**

Connectionist Temporal Classification

Connectionist temporal classification (CTC) algorithm can be used in a situation where **we do not have aligned data**.

Example:

- Let $\mathbf{O = [O_1, O_2, ..., O_t]}$ be the sequence of "***t***" input frames
- Let $\mathbf{W = [W_1, W_2, ..., W_n]}$ be the corresponding sequence on "***n***" output frames.

Since we do not have an alignment between "***O***" and "***W***":

- CTC takes into account **all possible ways** of aligning "***W***" to "***O***".

CTC Objective and Alignment

- Again our starting point *p(W|O)*
- We introduce a **hard alignment path** as a random variable:

$$Z = (z_t \in \mathcal{V}' | t = 1, \ldots, T)$$

- We consider the following equations

$$p(W|O) = \sum_Z p(Z, W|O)$$
$$= \sum_Z p(Z|O) p(W|Z, O)$$

Note that the SECOND-TERM p(W/Z,O) is EQUAL to UNITY, since if we remove the repeated characters in Z we will get back "W" with probability=1

Understanding CTC Objective and Alignment

- Again our starting point *p(W|O)*
- We introduce a **hard alignment path** as a random variable:

$$Z = (z_t \in \mathcal{V}' | t = 1, \ldots, T)$$

- We consider a set of all possible alignment paths representing *W.* We call this set as *Z(W)*
- We incorporate this random variable to *p(W|O)* as follows:

$$p(W|O) = \sum_{Z \in \mathcal{Z}(W)} p(Z|O)$$

- *Z* is a sequence and it is difficult to deal with. Similar to the attention case, we use a chain rule to further factorize this equation (mistake: should be 1:(t-1)):

$$p(W|O) = \sum_{Z \in \mathcal{Z}(W)} \prod_{t=1}^{T} p(z_t | z_{1:t}, O)$$

- This is not very difficult compared with an attention-based method, since ***Z* and *O* are based on the same length T**

- Note each "Z" is one alignment, which when repeated characters are removed give back "W"

Relation Between Definition 1 and Definition 2

Conditional probability Theorem

$$P\left(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}\right) = P\left(\boldsymbol{x}^{(1)}\right) \times P\left(\boldsymbol{x}^{(2)} \mid \boldsymbol{x}^{(1)}\right) \times \cdots \times P\left(\boldsymbol{x}^{(T)} \mid \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)}\right)$$

Definition 1

$$P(x^{(1)}, \ldots, x^{(T)}) = \prod_{t=1}^{T} P\left(\boldsymbol{x}^{(t)} \mid \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)}\right)$$

Definition 2

Example

Say there are 10 input frames corresponding to the word "**INDIA**".

Let $\mathbf{O = [O_1, O_2, ..., O_{10}]}$ and **Y=['I', 'N', 'D', 'I', 'A']**.

A few sample alignments considered by CTC are as follows:

	O_1	O_2	O_3	O_4	O_5	O_6	O_7	O_8	O_9	O_{10}
Align 1	I	I	N	N	N	D	I	A	A	A
Align 2	I	N	N	D	D	D	I	I	A	A
Align 3	I	I	I	N	D	I	I	A	A	A
Align 4	I	I	N	D	D	D	I	A	A	A
Align 5	I	I	I	I	N	D	I	A	A	A

Repetitions are merged to give the final output sequence: "***INDIA***"

CTC Objective

- Starting from $p(W|O)$
- Introduction of latent variable Z
- Factorized form:

$$
\begin{aligned}
p(W|O) &= \sum_{Z \in f^{-1}(W)} p(Z|O) \\
&= \sum_{Z \in f^{-1}(W)} \prod_{t=1}^{T} p(z_t|\cancel{z_{1:t-1}}, O) \\
&= \sum_{Z \in f^{-1}(W)} \prod_{t=1}^{T} p(z_t|O)
\end{aligned}
$$

- We are using probability chain rule
- The conditional assumption of z_t not depending on previous z_t is a “drawback” of CTC.. This is an approximation we make in CTC objective

Relation Between Definition 1 and Definition 2

Conditional probability Theorem

$$P\left(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}\right) = P\left(\boldsymbol{x}^{(1)}\right) \times P\left(\boldsymbol{x}^{(2)} \mid \boldsymbol{x}^{(1)}\right) \times \cdots \times P\left(\boldsymbol{x}^{(T)} \mid \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)}\right)$$

Definition 1

$$P(x^{(1)}, \ldots, x^{(T)}) = \prod_{t=1}^{T} P\left(\boldsymbol{x}^{(t)} \mid \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)}\right)$$

Definition 2

Example

Say there are 10 input frames corresponding to the word "**INDIA**".

Let $\mathbf{O = [O_1, O_2, ..., O_{10}]}$ and **W=['I', 'N', 'D', 'I', 'A']**.

A few sample alignments considered by CTC are as follows:

	O_1	O_2	O_3	O_4	O_5	O_6	O_7	O_8	O_9	O_{10}
Align 1	I	I	N	N	N	D	I	A	A	A
Align 2	I	N	N	D	D	D	I	I	A	A
Align 3	I	I	I	N	D	I	I	A	A	A
Align 4	I	I	N	D	D	D	I	A	A	A
Align 5	I	I	I	I	N	D	I	A	A	A

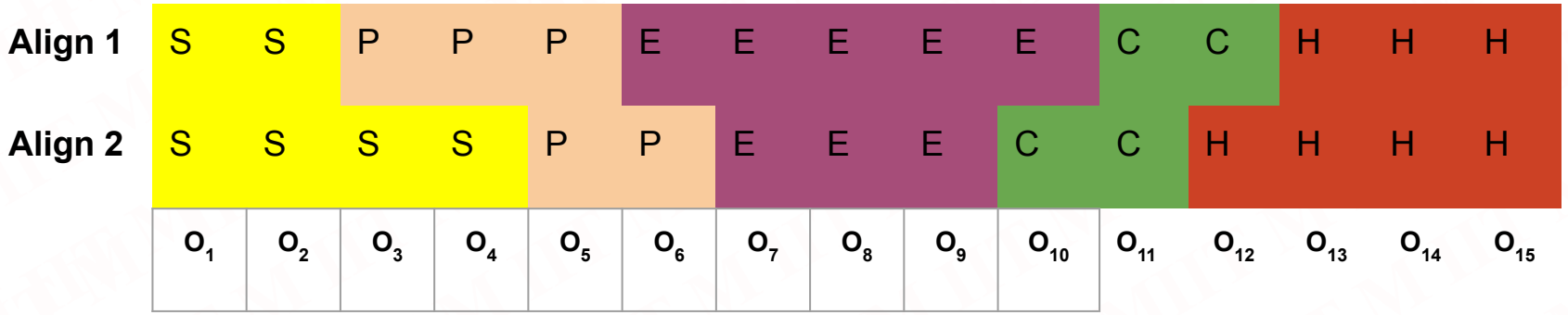
Repetitions are merged to give the final output sequence: "***INDIA***"

Handling Repeated Characters

Is the previous method of collapsing the alignments will work for repeated characters also (For example: "SPEECH")?

Example:

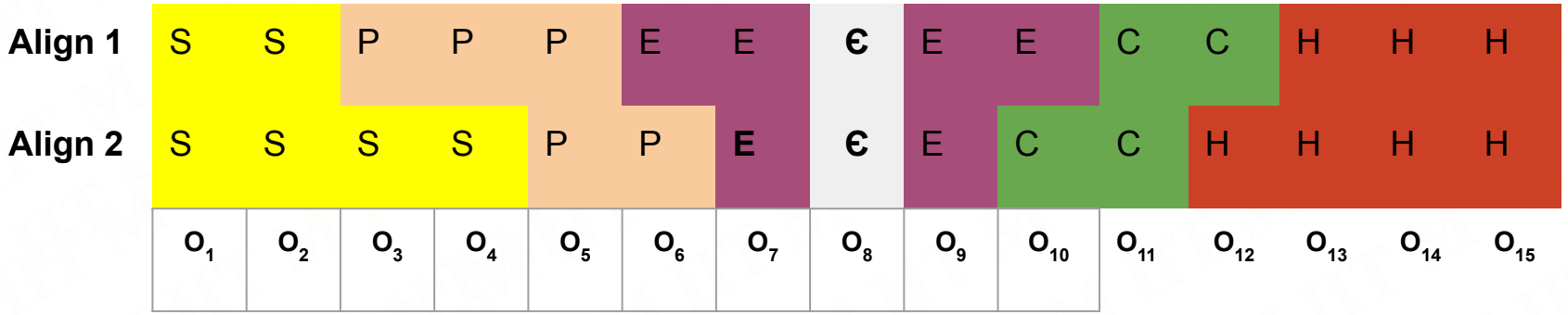
Let **O=[O_1, O_2 ,…., O_{15}]** and **W=['S', 'P', 'E', 'E', 'C', 'H']**



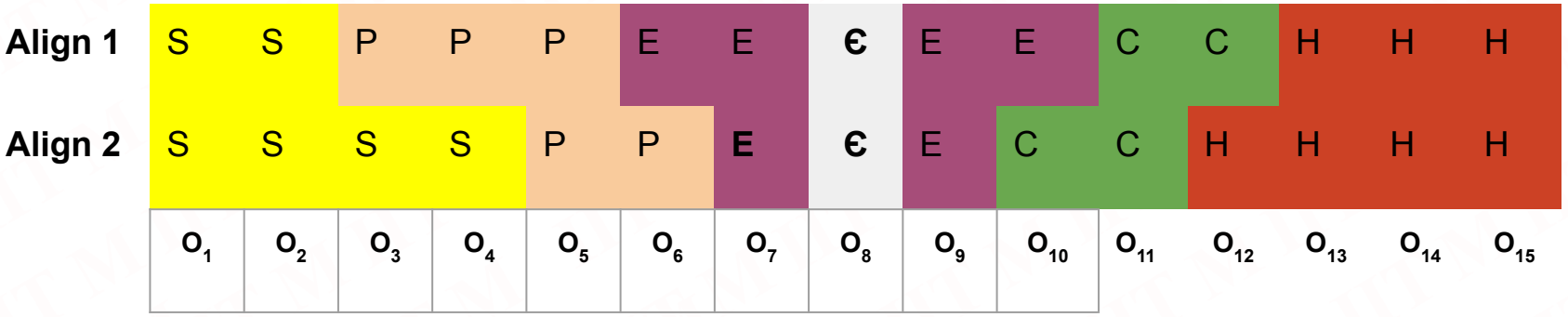
Both **Alignment 1** and **Alignment 2** when collapsed give: "**SPECH**" (**which is wrong**)

Handling Repeated Characters

To handle this problem we can **introduce blank (Є or -) between repeated characters**.



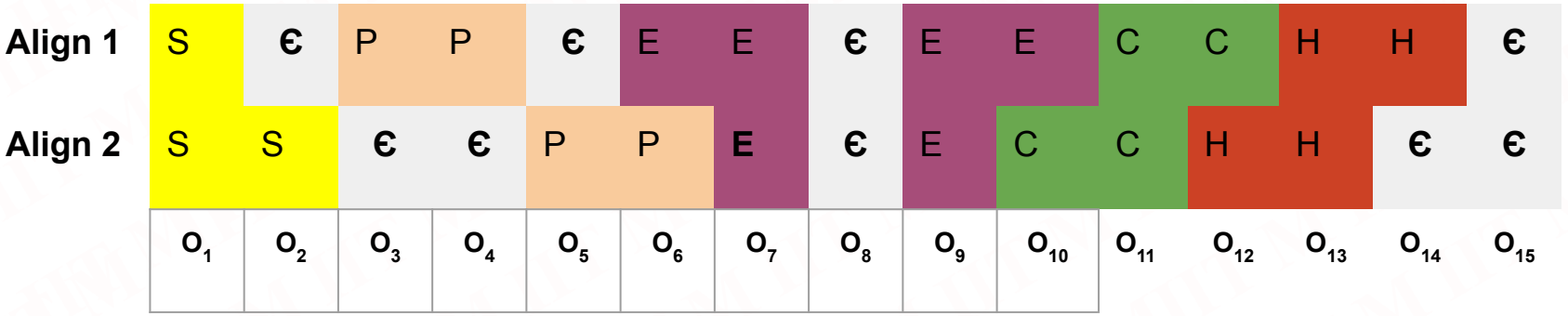
Handling Repeated Characters



- So First **we will collapse all the repeated characters in the alignments** (For both the alignments the collapsed sequence will be {'S', 'P', 'E', 'Є', 'E', 'C', 'H'}
- The Second step will be **to remove the blanks** (Є) to get the desired output (SPEECH)

Properties of Blank Symbol

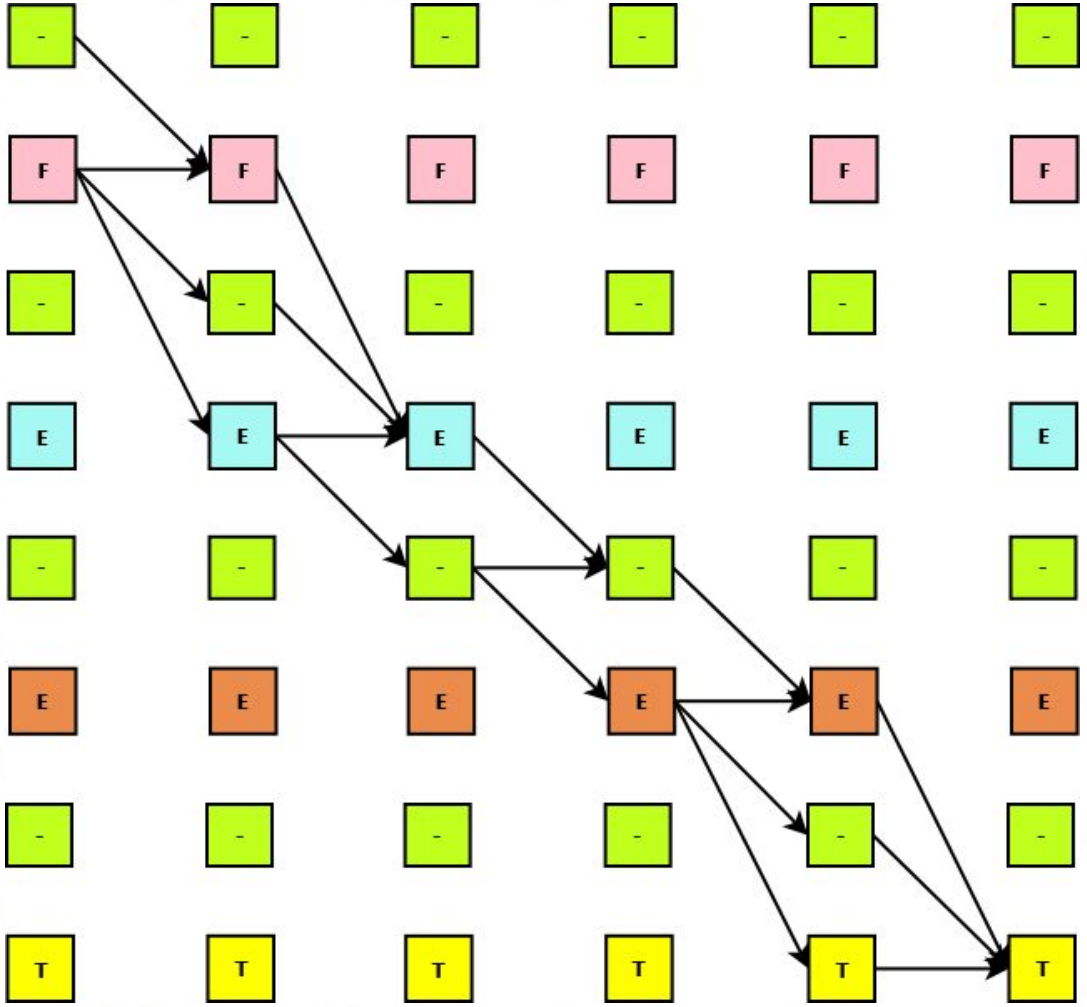
- It's **not always necessary to align every frame to a output symbol.**
- In Speech Recognition, for example, the **input can have stretches of silence** with no corresponding output.
- **Blank symbols can also be used to represent silence regions.**



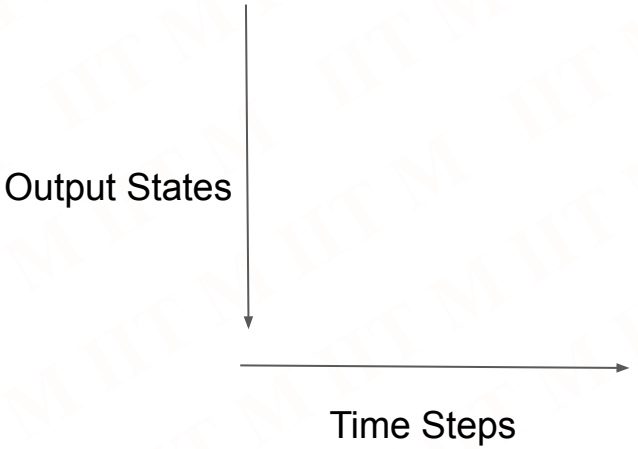
Properties of CTC Alignments

- First, the allowed alignments between ***O*** (input) and ***W*** (output) are **monotonic.** If **we advance to the next input, we can keep the corresponding output the same or advance to the next one**.
- A second property is that the **alignment of *O* to *W* is many-to-one.** One or more input elements can align to a single output element but not vice-versa**.**
- The length of ***W*** **cannot be greater than the length of *O***.

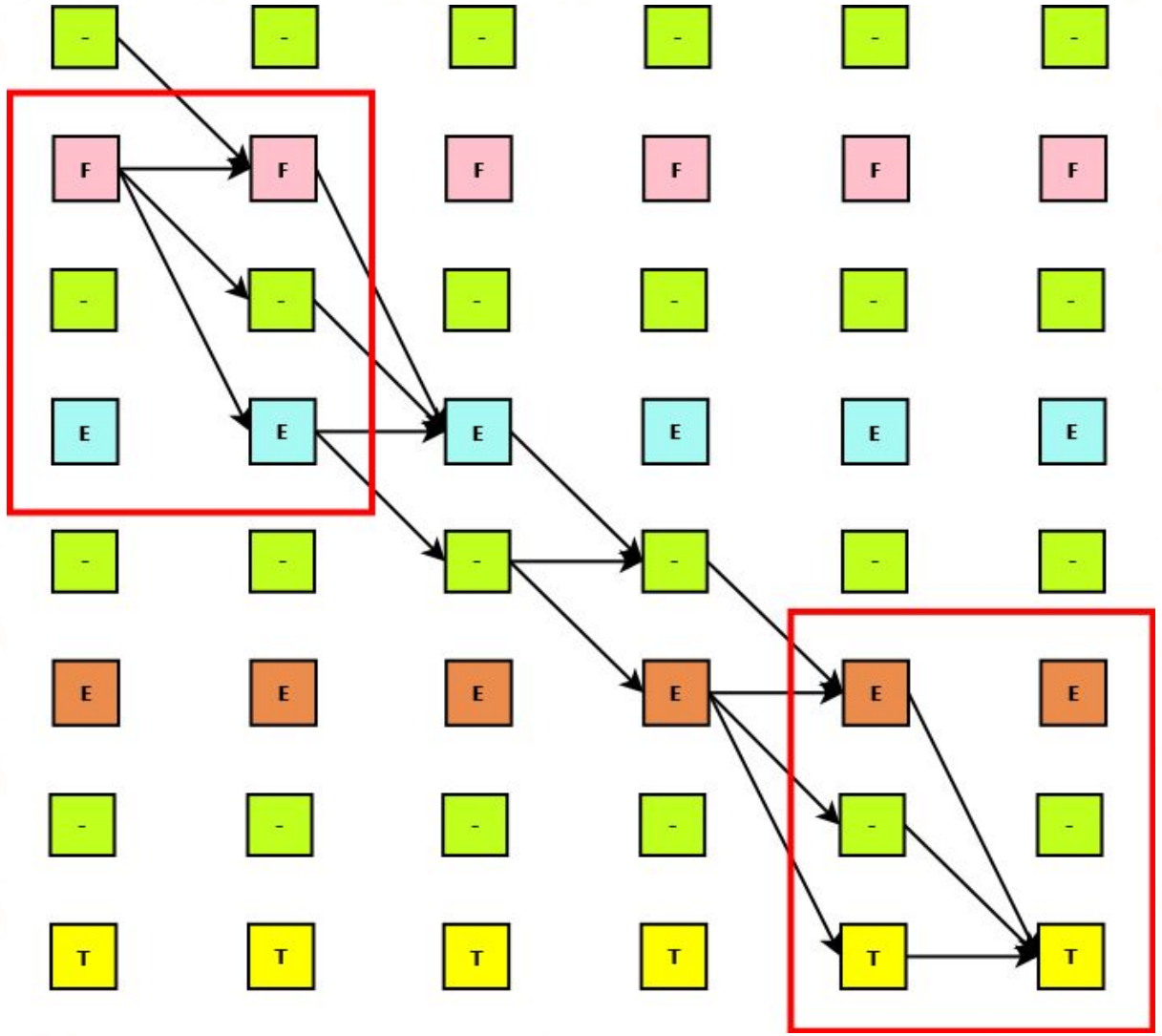
Monotonicity in CTC



- Let the target word be : “FEET”
- Let there be six input frames.



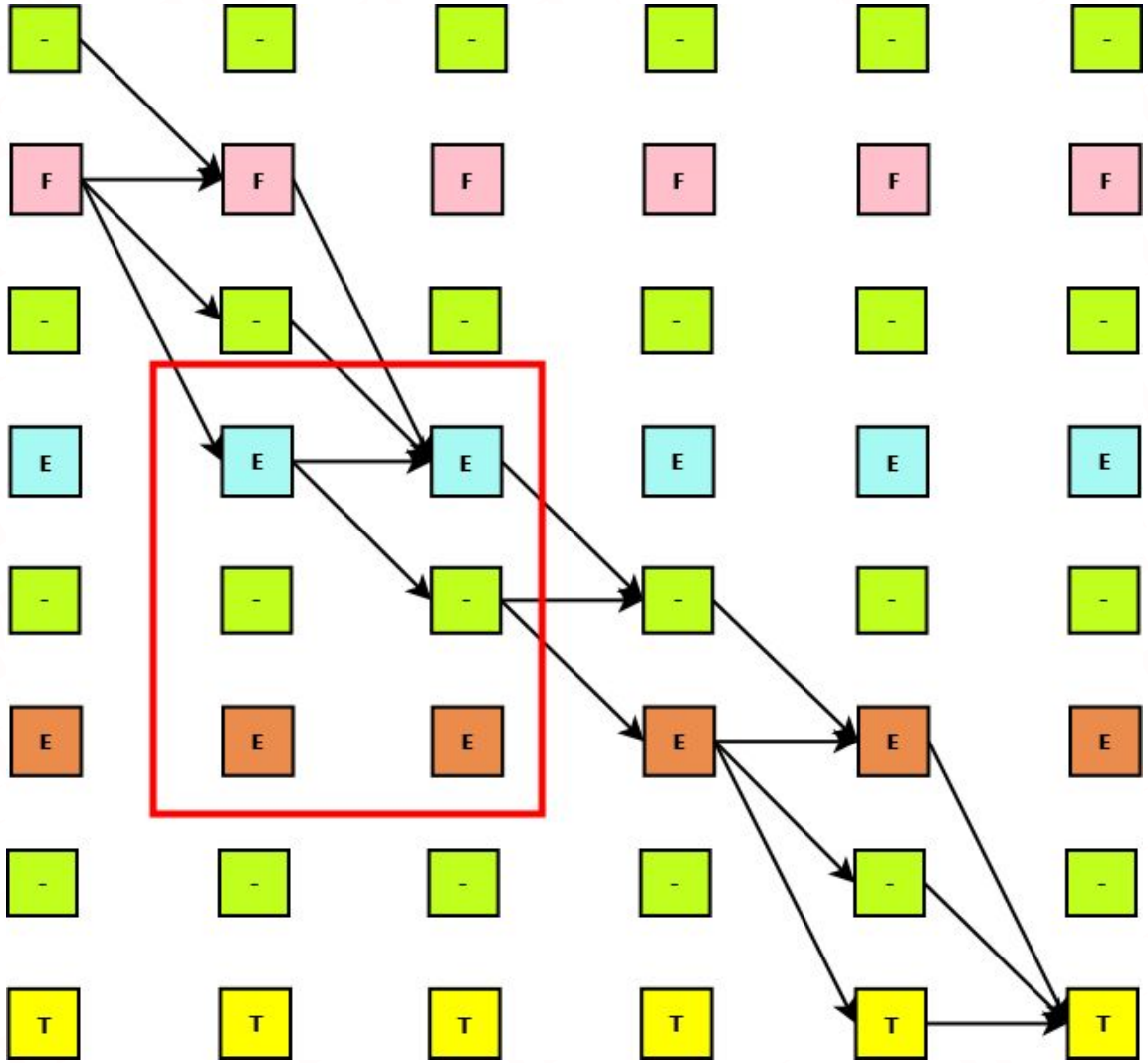
Monotonicity in CTC



If the **next state is not the repeated state** in the output sequence, i.e., transition form **F -> E** or **E -> T**, then

- We can continue to **remain in the same state**, or
- We can **transition to a blank symbol**, or
- We can **transition to the next state**

Monotonicity in CTC



If the next state is **the repeated state in the output sequence**, i.e., transition form **E -> E**, then

- We can continue to **remain in the same state**, or
- We can **transition to a blank symbol**, or

We CANNOT skip the blank state.

CTC Loss

Let's ***X*** represent the **sequence of audio frame** and ***Y*** represent the **sequence of output tokens** (characters)

The CTC Loss (***L***) can be defined as :

CTC conditional probability

$$L = \sum_{(X,Y)\in\mathcal{D}} -\log p(Y \mid X)$$

Training Data

CTC Conditional Probability

Let ***A*** represent a **valid alignment (which satisfy the properties of CTC alignment)** such that :

$$A = \{a_1 \ , \ a_2 \ , \ a_3 \ , \ \dots \ , \ a_T\}$$

So **CTC conditional probability** ($p(Y \mid X)$) can be defined as :

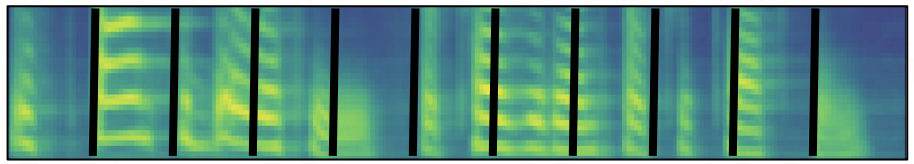
$$p(Y \mid X) = \sum_{A \in \mathcal{A}_{X,Y}} \prod_{t=1}^{T} p_t(a_t \mid X)$$

Marginalizes over the set of valid alignments

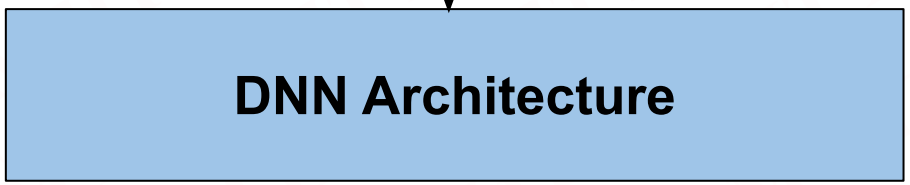
computing the probability for a single alignments

Training CTC

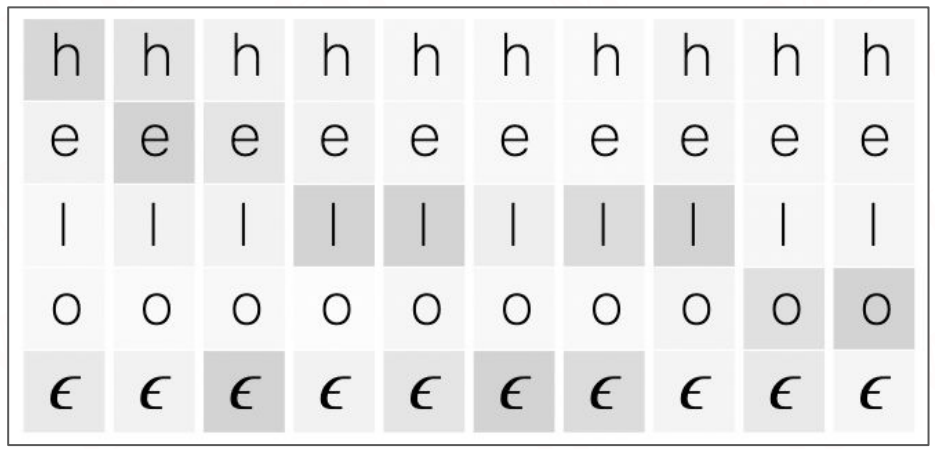
Let ***X*** be the **input frame sequence** and ***Y*** be the output sequence (“***hello”***)



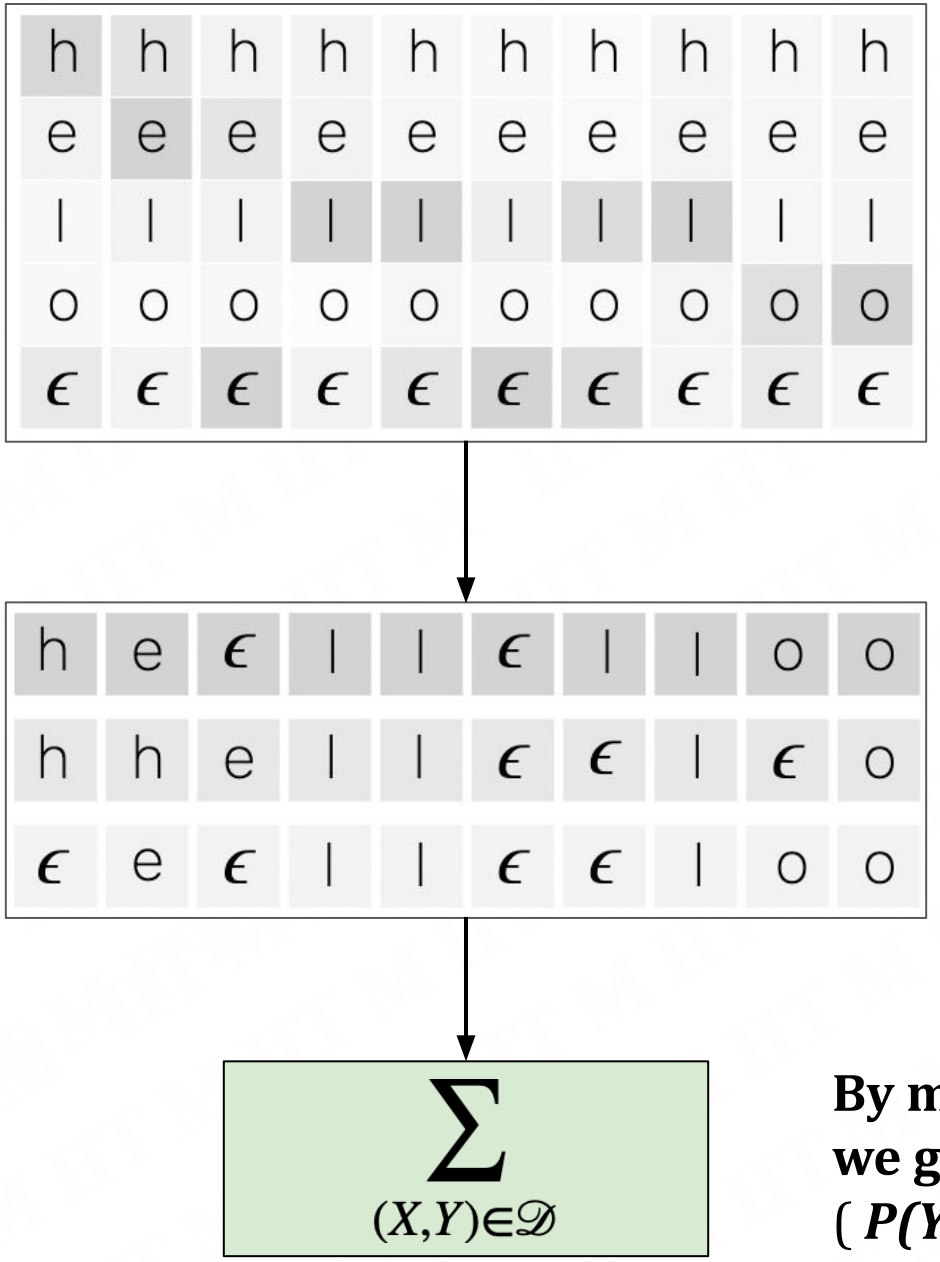
We start with an **input sequence like a spectrogram** of a audio



The input is fed into a DNN



The **network gives $p_t(a|X)$, a distribution over the outputs {h, e, l, o, ϵ}** (using softmax). The **darker shades represent larger probability compare to other labels.**



With the **per time-step output distribution** (that we obtain in the previous step), we c**ompute the probability of different sequence** (alignments).

By marginalizing over alignments, we get a distribution over outputs.
($P(Y \mid X)$)

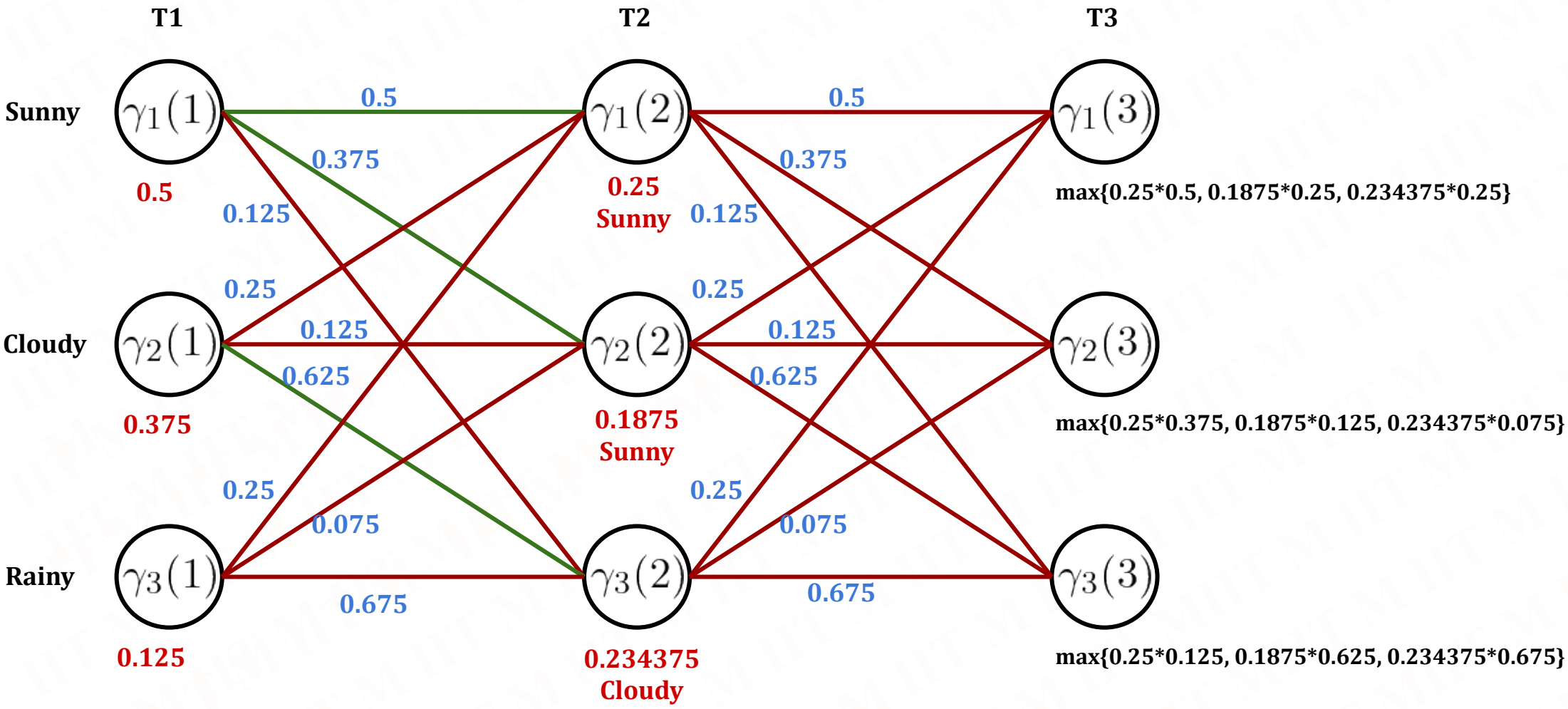
Training CTC: Using Forward-Backward Algorithm

- Even with the **monotonicity property, there will be multiple ways of aligning the output sequence to the input sequence.**
- So how do we keep **track of the probabilities of generating each of the possible alignments** ?
- We can use **Forward-Backward Algorithm** similar to what we have discussed in HMM

Recall Viterbi Algorithm - Efficient way to find best path

- In general, if there are **N states** and **the length of the sequence is T**, then the total **number of possible paths is N^T**
- As total no. of paths is very large, this process is expensive.
- There is an effective algorithm to find the most probable path in very less time complexity.
- The algorithm is known as **Viterbi Algorithm**. Please note it is **not an approximation.**
- **Reduces the computation to: N^2 x T instead of N^T**
-

Viterbi Algorithm - from “t” to “t+1” – only N^2 computes



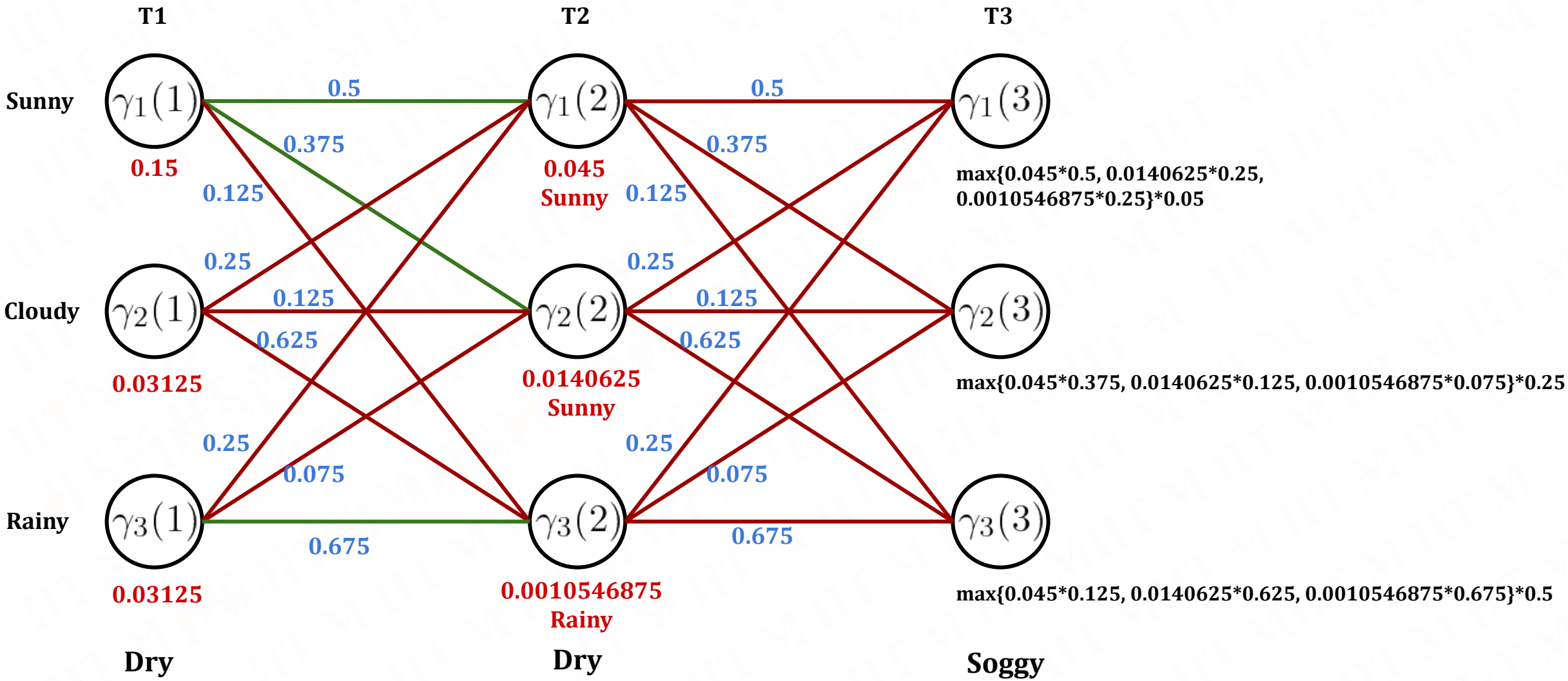
The CTC Forward Backward Algorithm

- Recall

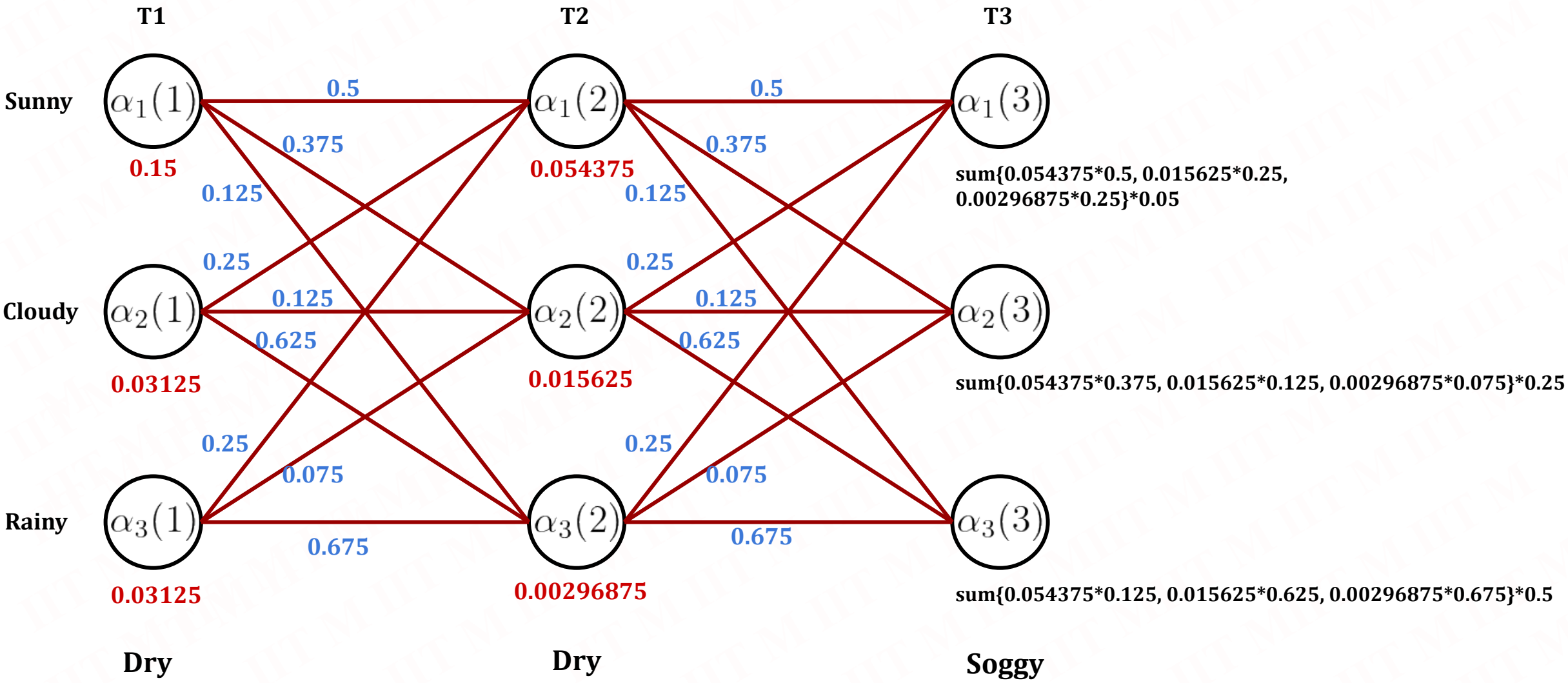
$$p(Y \mid X) = \sum_{A \in \mathcal{A}_{X,Y}} \prod_{t=1}^{T} p_t(a_t \mid X)$$

- Challenge : sum over all paths corresponding to a given labelling (char. Sequence)
- Key idea is:
  - The sum over paths corresponding to a labelling **can be broken down into an iterative sum** over paths corresponding to prefixes of that labelling
- Iterations can be efficiently computed over forward and backward variables

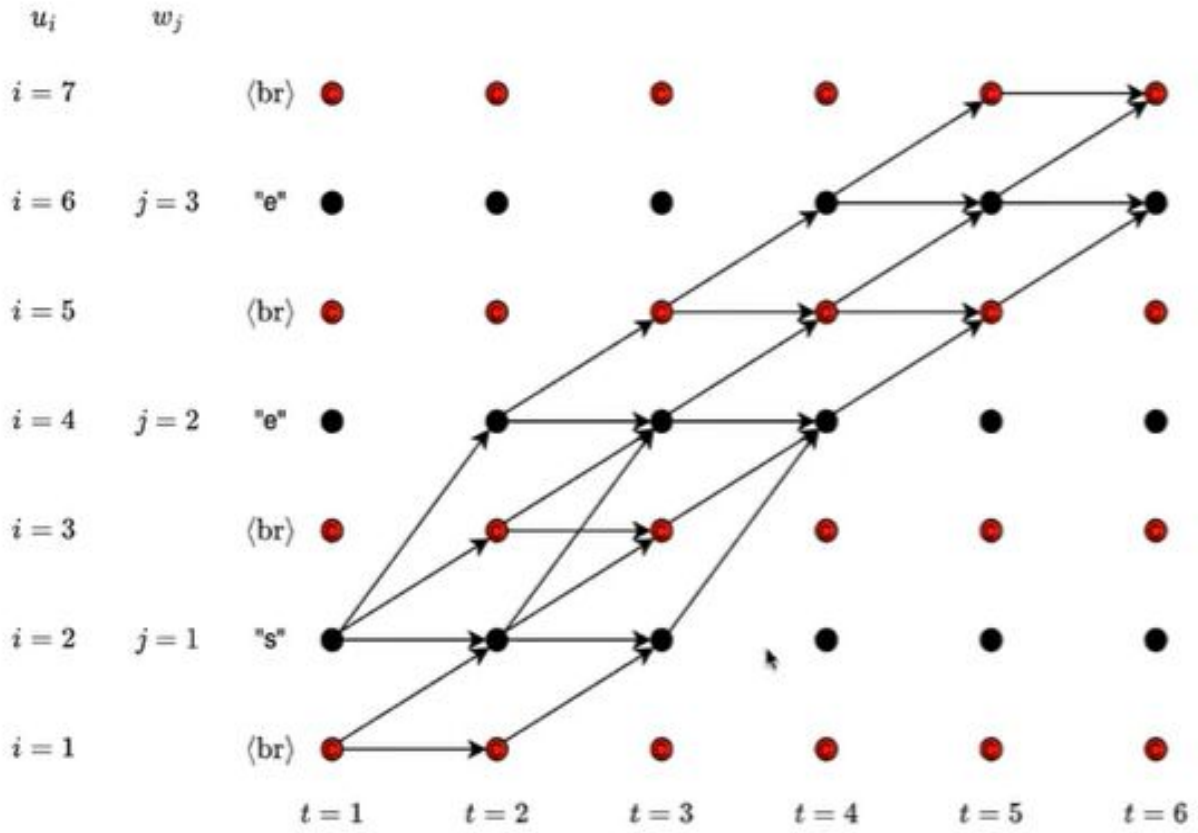
Viterbi Algorithm For HMM - Weather Example



Forward Algorithm For HMM - N^2 computes from t to (t+1)



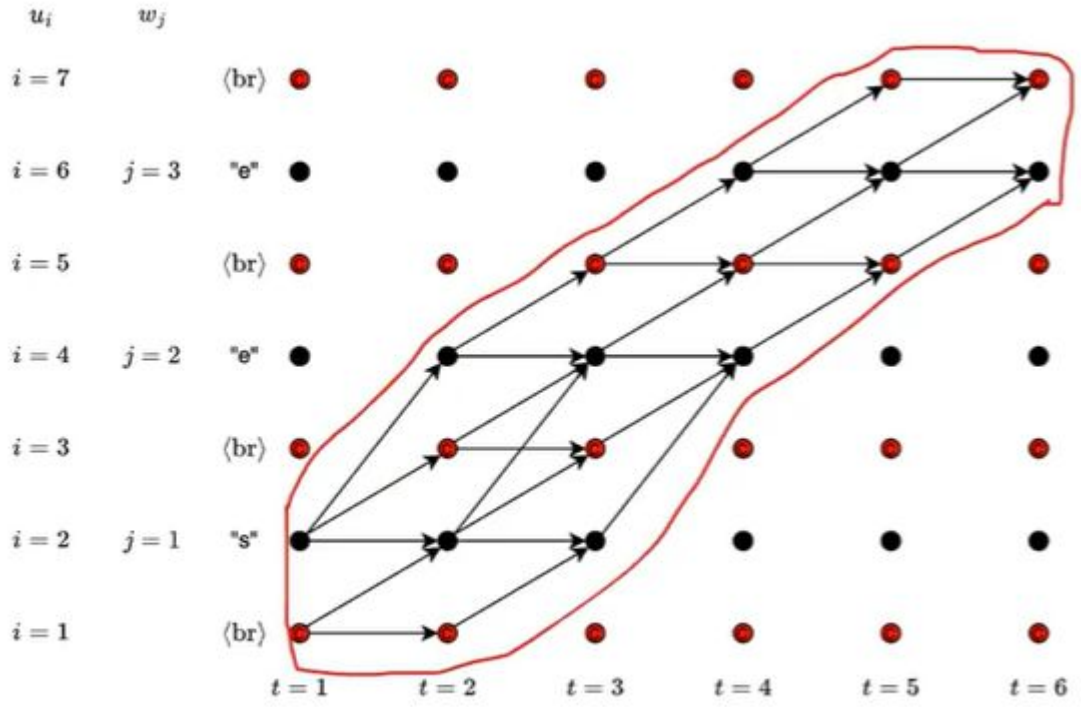
CTC possible alignment Paths



- Token sequence $W = (w_1, w_2, \ldots, w_j, \ldots, w_J)$
- New token sequence $U = (u_1, u_2, \ldots, u_i, \ldots, u_{2J+1})$ where u_i =
, when i is an odd number

- Note at each time-step, the paths can also be at “blank tokens” or a valid character token

Recall the CTC objective

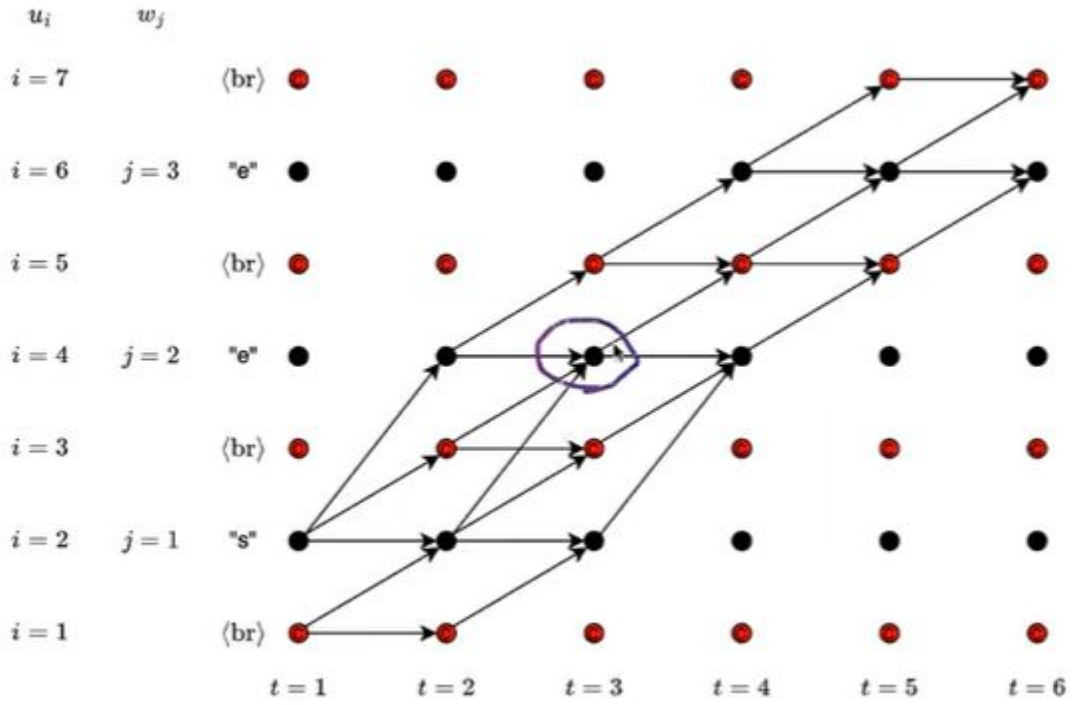


- Summation over $Z \in f^{-1}(W, O)$
- All possible alignment sequences Z that represent W

$$p(W|O,\Theta) = \sum_{Z \in f^{-1}(W,O)} p(Z|O,\Theta)$$

- Summation of the region surrounded by a red line

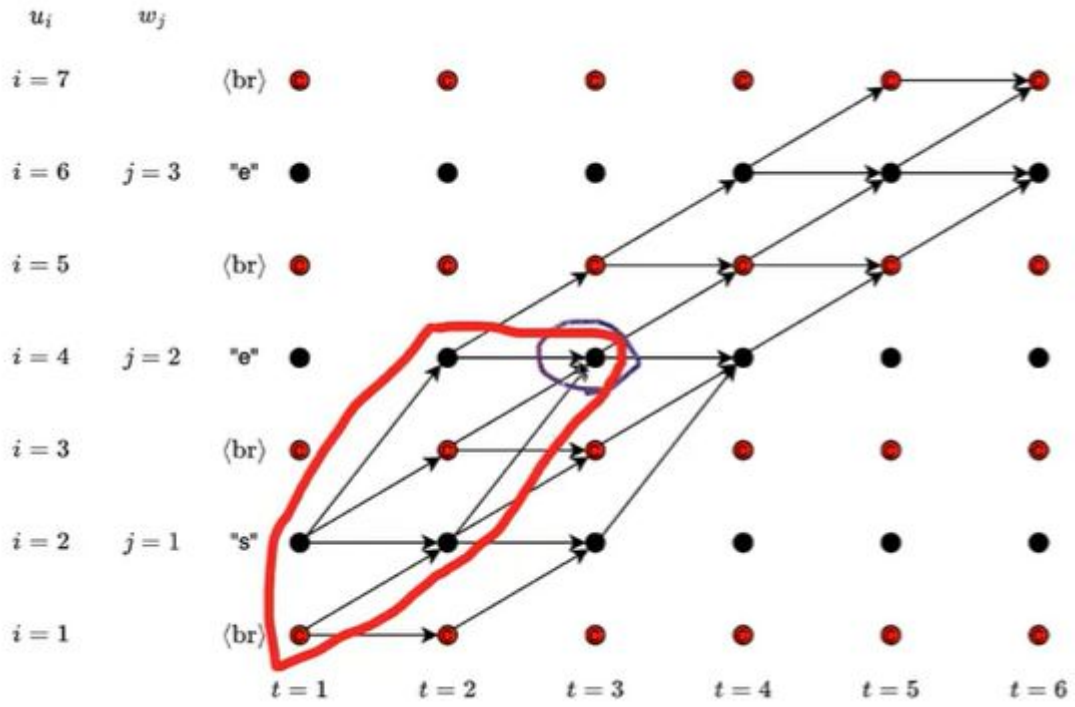
CTC Forward Backward Variable



- We focus on the occupation probability of one point, e.g., $\gamma(3,4)$
  - $t = 3$ and $i = 4$
- How to represent this probability?
  - Consider the all possible paths from the beginning ($t = 1$) to $\gamma(3,4)$
  - Consider the all possible paths the beginning from $\gamma(3,4)$ to the end ($t = T$)

- Calculating this occupation probability will involve forward-backward variables
- In CTC case, we will do it for “i” which includes blank characters

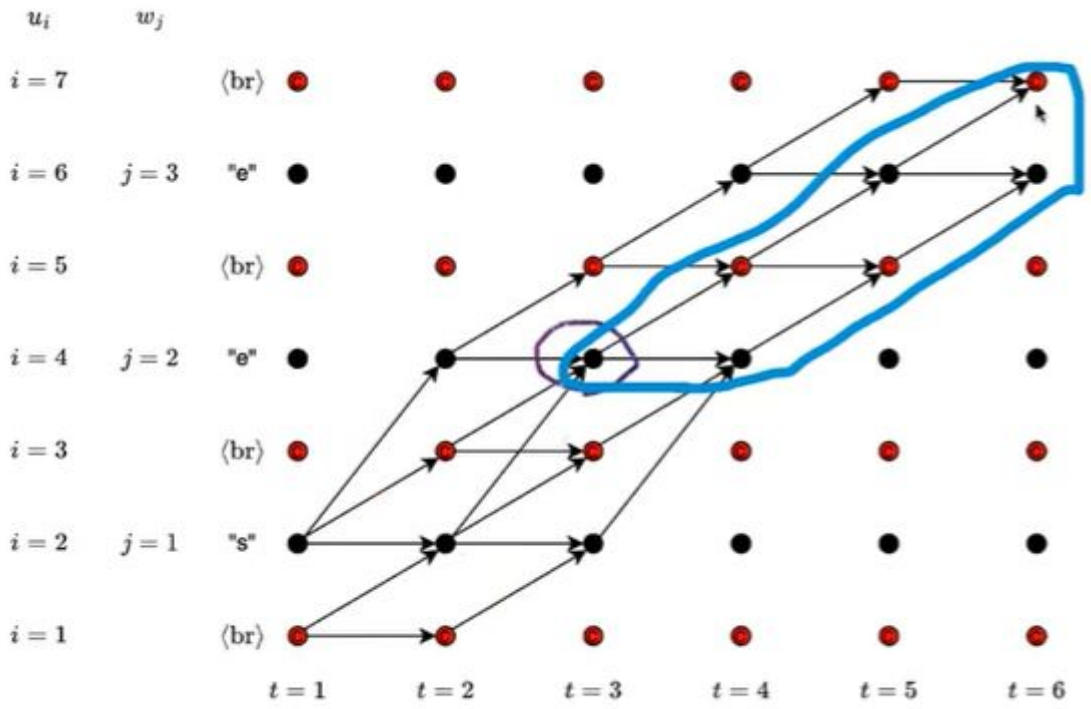
CTC Forward Variable Path



- We focus on the occupation probability of one point, e.g., $\gamma(3,4)$
  - $t = 3$ and $i = 4$
- How to represent this probability?
  - Consider the all possible paths from the beginning ($t = 1$) to $\gamma(3,4)$

- These are ALL PATHS from (t=1,i=1) to t=3,i=4 for this case. We can also start from t=1,i=2 and we count those paths also upto t=3,i=4
- In general: All paths upto time time="t" and "token=i"

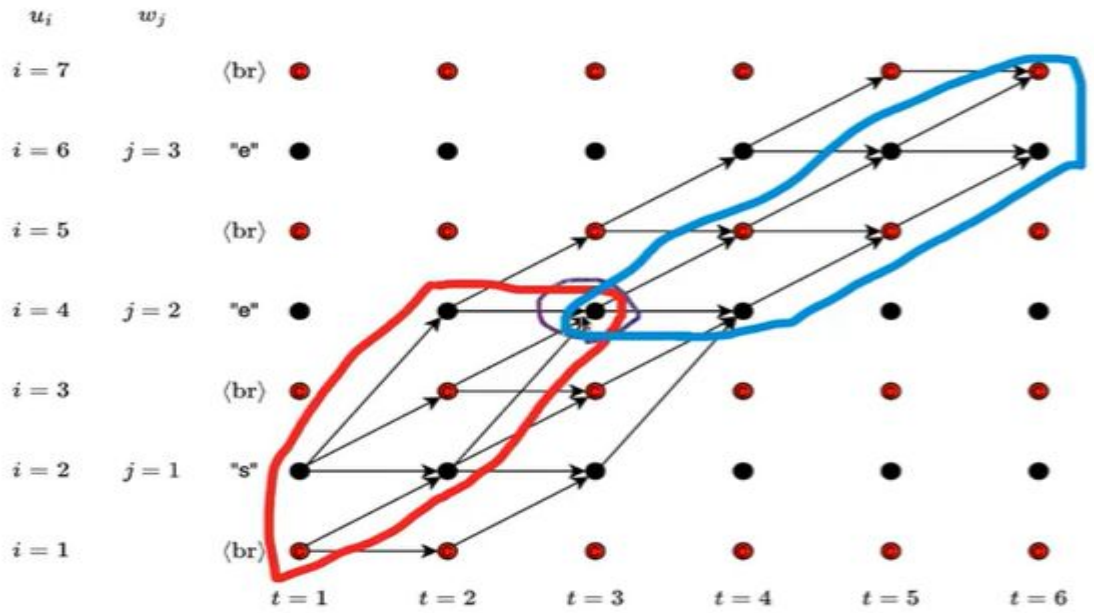
CTC Backward Variable Path



- We focus on the occupation probability of one point, e.g., $\gamma(3,4)$
  - $t = 3$ and $i = 4$
- How to represent this probability?
  - Consider the all possible paths the beginning from $\gamma(3,4)$ to the end $(t = T)$

- In this case: we look all paths beginning from t=3,i=4 to the last time-step t=T
- It includes all paths that end at i=7 (blank token) or i=6 a valid character, but at time t=T
- In general case: from all paths from (t,i) to last time stamp t=T

Finding Occupation Probability

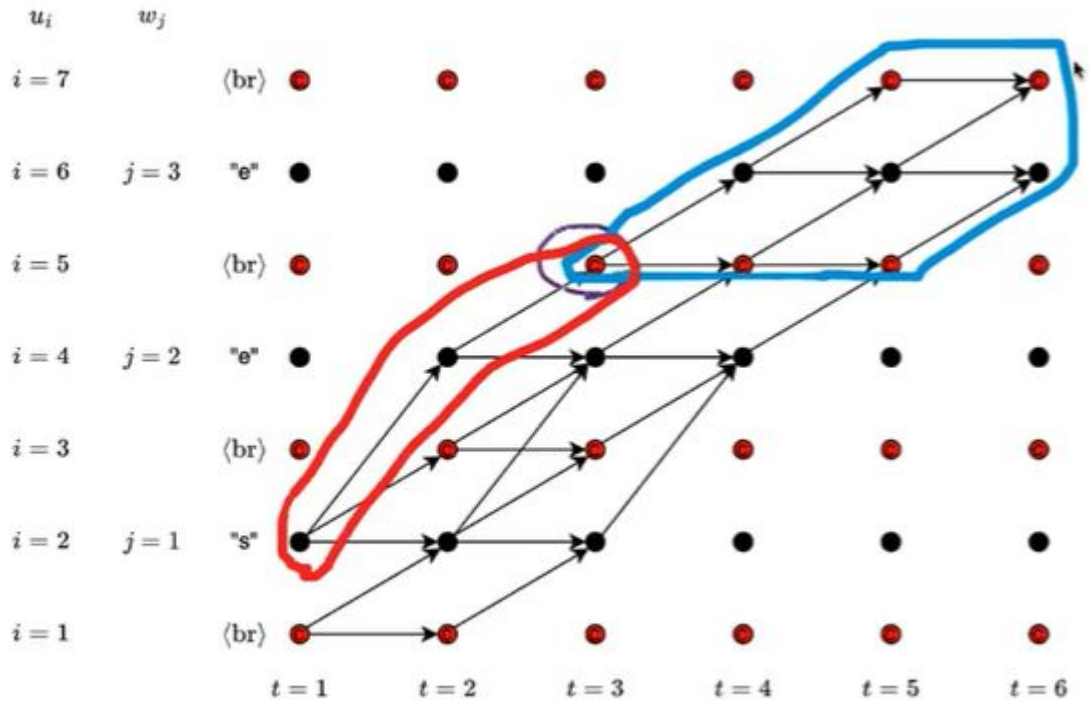


- We focus on the occupation probability of one point, e.g., $\gamma(3,4)$
  - $t = 3$ and $i = 4$
- How to represent this probability?
  - Consider the all possible paths from the beginning ($t = 1$) to $\gamma(3,4)$
  - Consider the all possible paths the beginning from $\gamma(3,4)$ to the end ($t = T$)
- This is represented by

$$\gamma(t,i) = \sum_{Z_{1:t-1}, Z_{t+1,T}} p(Z_{1:t-1}, z_t = u_i, Z_{t+1,T} | O, \Theta)$$

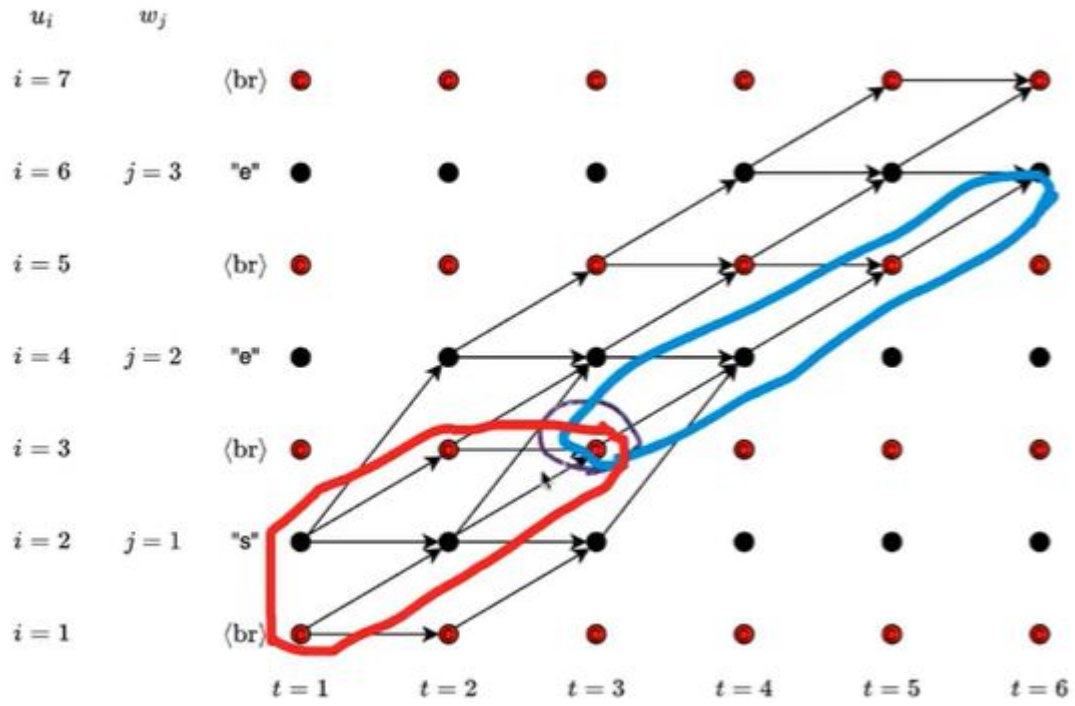
- Note at (t,i), I need to find all paths from beginning to z_t=u_i and
- All paths from (t,i) to end

Occupation Probability at another Node



- We focus on the occupation probability of one point, e.g., $\gamma(3,5)$
  - $t = 3$ and $i = 5$

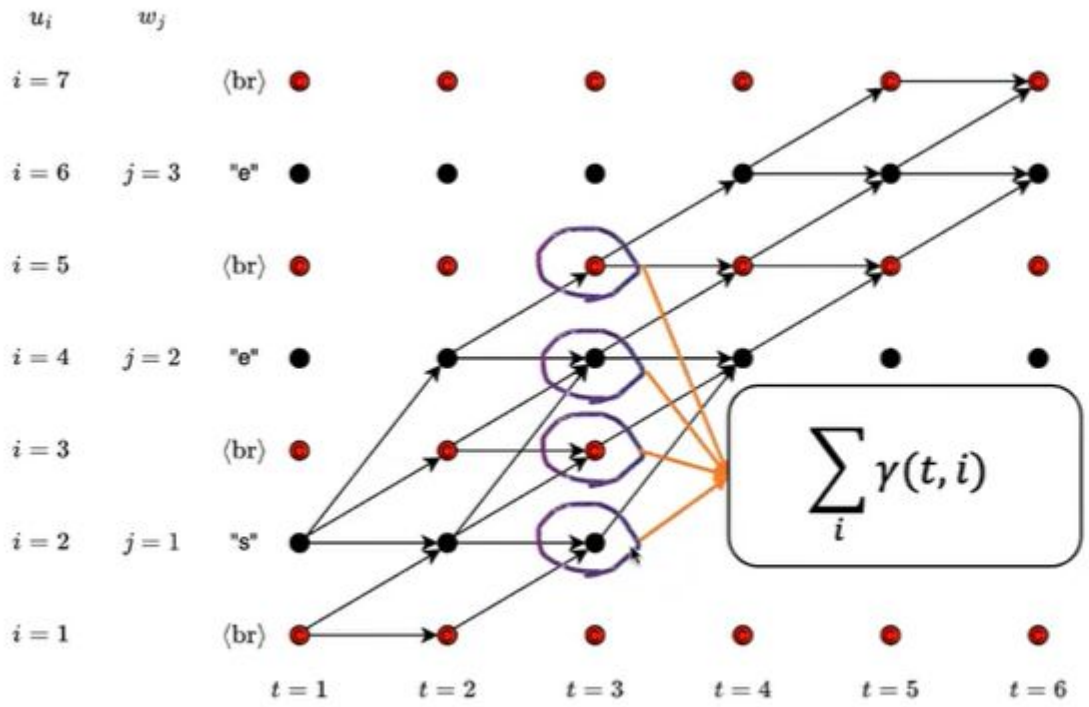
Occupation Probability at yet another Node



- We focus on the occupation probability of one point, e.g., $\gamma(3,3)$
  - $t = 3$ and $i = 3$

- These paths indicate “subsequences” upto that point

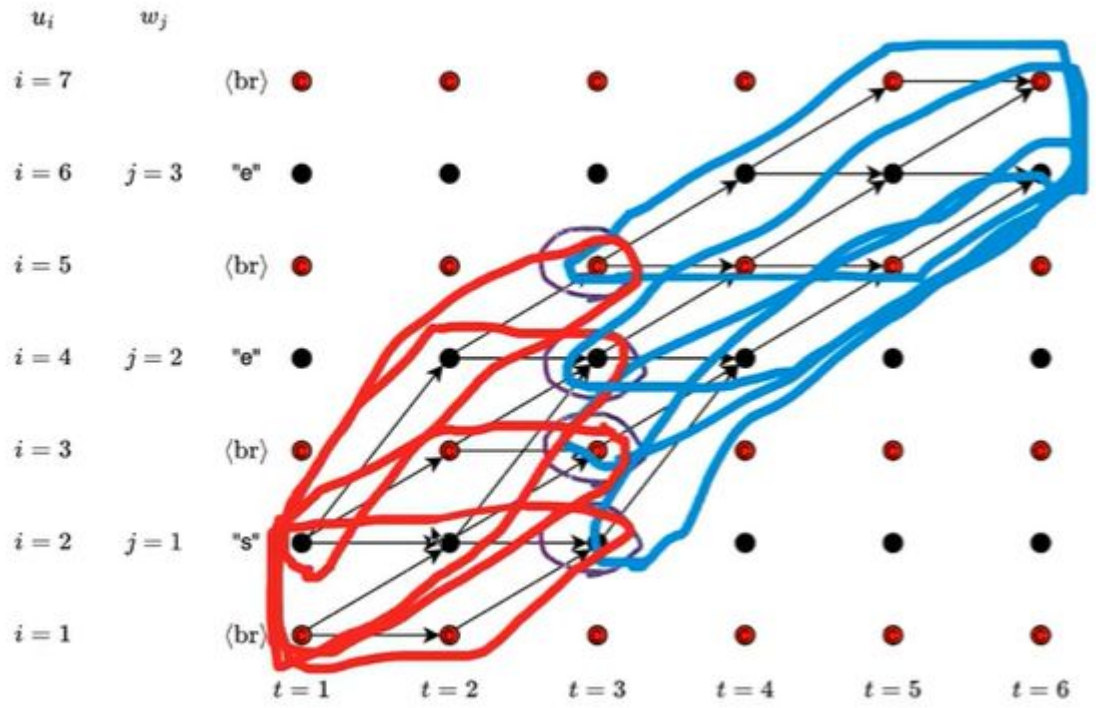
What does adding all “gamma” at a given time “t” give?



- What happens if we add all forward-backward variables across the y-axis?
- $\sum_i \gamma(t, i) = ???$

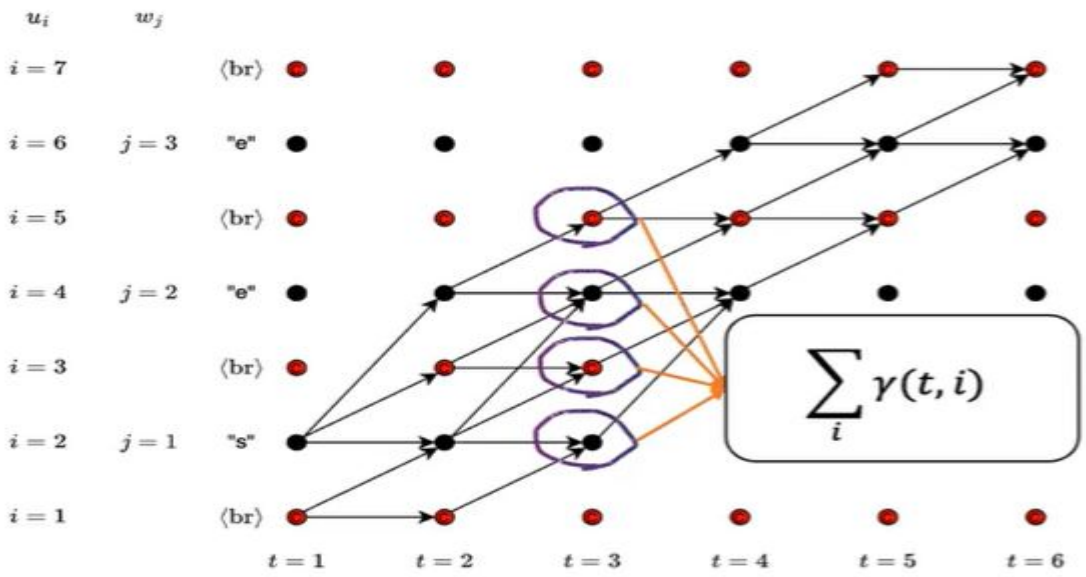
- Hint: it is looking all paths from beginning, and ALL paths from “t’ to END

Consider all Forward Backward Paths at time “t”



- What happens if we add all forward-backward variables across the y-axis?
- $\sum_i \gamma(t, i) = ???$

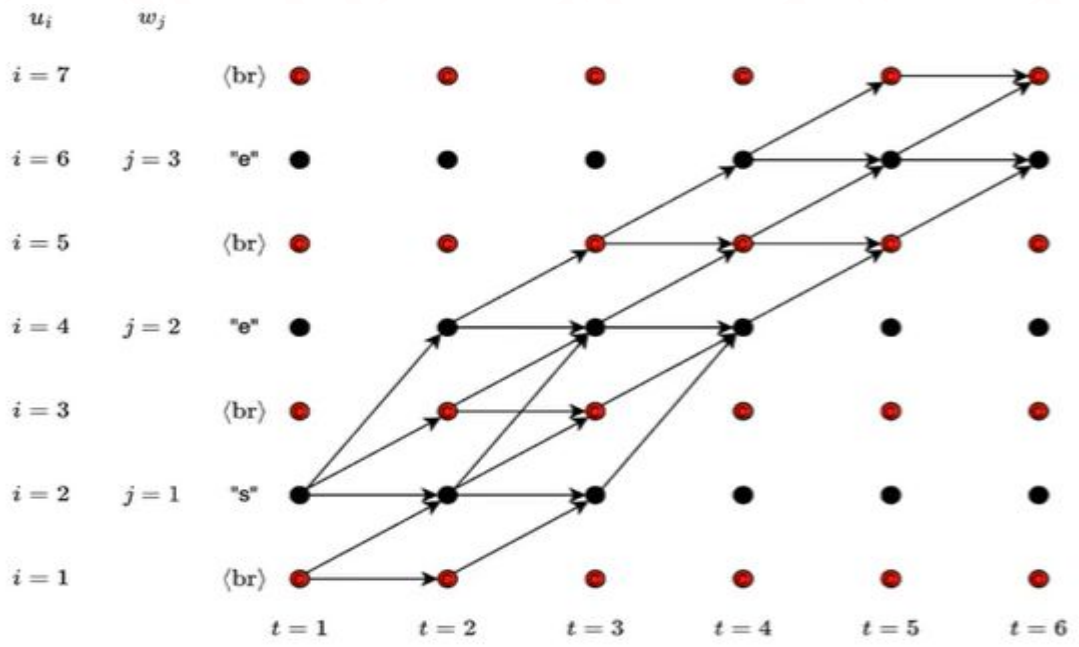
This gives TOTAL probability



- What happens if we add all forward-backward variables across the y-axis?
- $\sum_i \gamma(t,i) = p(W|O,\Theta)$
- Similar to the HMM EM algorithm, $\gamma(t,i)$ is computed by the forward-backward algorithm
- We can avoid to compute $\sum_Z$

Note here we are using idea of Forward-Backward Variable to find total P(W/O,\theta)
This is different from finding all possible alignment paths (i.e. z) and summing over all of those “z paths”
This is an EFFICIENT way to find.. Similar trick to “Viterbi” where we are finding “best path” after finding all possible paths

CTC – Forward Backward – same label multiple times at "t"



- What is the relationship between w and u_i?
- $w \in$ {"
," "e," "s"}

- In each t, the same token occurs multiple times
  - $t = 3$,
 occurs at $i = 3, 5$
  - $t = 4$, "e" occurs at $i =???$
- Therefore, if we connect w and $\gamma(t, i)$, we can just add $\gamma(t, i)$ that occurs the same token w, i.e.,

$$\sum_{i:u_i=w} \gamma(t, i)$$

CTC Loss Function

- Let's use $p(W|O)$ as an objective function!

$$\hat{\Theta} = \text{argmax}_{\Theta}\, p(W|O,\Theta)$$

- Like the maximum likelihood EM algorithm
- Note that we often use the loss and take the logarithm, in this case

$$\hat{\Theta} = \text{argmin}_{\Theta}\, -\log p(W|O,\Theta)$$

- However, we could no longer use the EM algorithm due to the complicated function form (we could not find the closed-form solution).

- Let's use the gradient descent (in the loss case)

Derivative of CTC Loss

- Based on the chain rule, the derivative is factorized with three components

$$\frac{\partial \mathcal{L}^{\text{ctc}}(\Theta)}{\partial \Theta} = \frac{\partial \mathcal{L}^{\text{ctc}}(\Theta)}{\partial y_{wt}} \frac{\partial y_{wt}}{\partial \Theta}$$

$$= \frac{\partial - \log p(W|O,\Theta)}{\partial y_{wt}} \frac{\partial y_{wt}}{\partial \Theta}$$

$$= -\frac{1}{p(W|O,\Theta)} \frac{\partial p(W|O,\Theta)}{\partial y_{wt}} \frac{\partial y_{wt}}{\partial \Theta}$$

We use chain rule for the derivative

- We rewrite $p(W|O,\Theta)$ so that we can take the derivatives of $p(W|O,\Theta)$ with respect to y_{wt}

- $y_{wt} = [p(w|O,\Theta)]_t$
  - This is typically obtained by a neural network, i.e.,
  - $y_{wt} = p(w|\mathbf{h}_t,\Theta) = [\text{softmax}\,(\mathbf{W}\mathbf{h}_t + \mathbf{b})]_w$
  - Softmax probability for each t and each w
  - We will explain it more details later .

CTC Loss Objective Using Forward Backward

$$\frac{\partial \mathcal{L}^{\text{ctc}}(\Theta)}{\partial \Theta} = \frac{\sum_{i:u_i=w} \gamma(t,i)}{p(W|O,\Theta)} \left(-\frac{1}{y_{wt}} \frac{\partial y_{wt}}{\partial \Theta} \right)$$

Weighted summation of the occupation probability for token w at time i

Solved by a normal neural network backpropagation

- Let's focus on obtaining $\gamma(t,i)$ by using the forward-backward algorithm!!!

Why CTC for E2E-ASR?

- We can use “encoder” only architecture for CTC
  - Recall in Encoder-Decoder, Decoder was autoregressive
  - Alignment in Encoder-Decoder came through cross-attention – too much flexibility?

- Encoder-only can “parallelise” forward-pass – so fast

- Monotonicity is enforced
  - Easier to do “real-time” causal inferencing

CTC Encoder only Softmax

CTC encoder $p(z_t|O)$

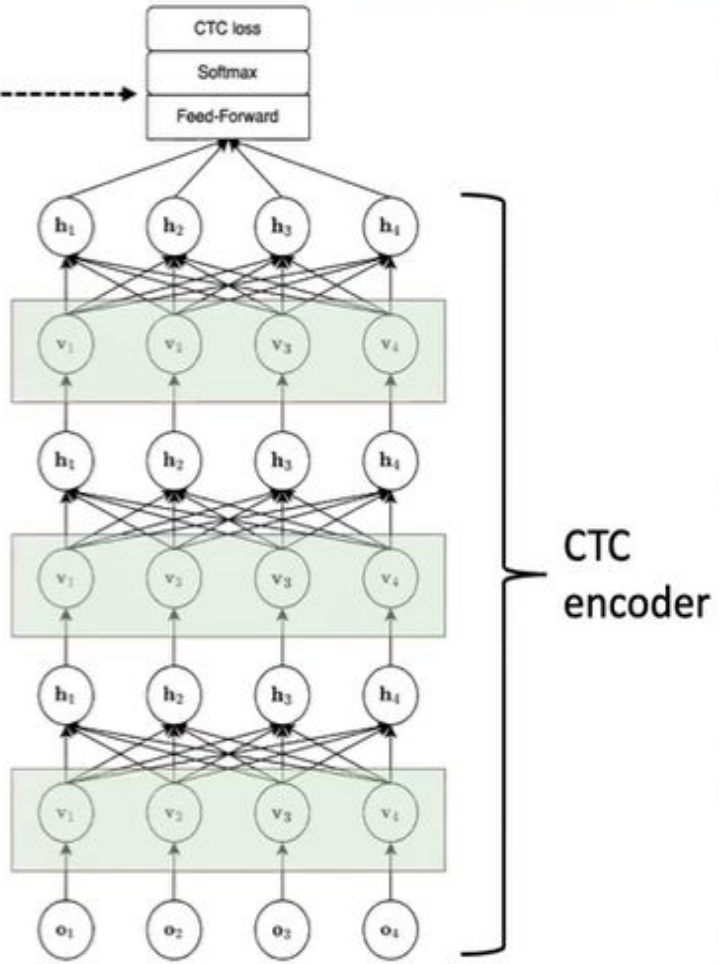
- We can use any of the encoder architectures I introduced in my course
  - BLSTM, CNN, Transformer, Conformer, Branchformer, etc, to obtain D-dimensional vector $\mathbf{h}_t$

    $\mathbf{h}_t = \text{Encoder}(O) \in \mathbb{R}^D$
  - The improvement of the encoder architecture directly affects to the entire ASR performance
    - Easy to develop a new architecture

- Then, add the linear layer and perform softmax

$$p(z_t|O) = \text{softmax}(\mathbf{W}\mathbf{h}_t + \mathbf{b})$$

- $\mathbf{W} \in \mathbb{R}^{(|\mathcal{V}|+1)\times D}$, $\mathbf{b} \in \mathbb{R}^{|\mathcal{V}|+1}$ ← Linear transform
- Convert it to the token vocabulary dimension $|\mathcal{V}| + 1$ (← blank symbol)



From Shinji's videos: WAV LAB CMU

CTC Conditional Independence

- No token dependency due to the conditional independence assumptions
  - **Pros**
    - The model becomes simple
    - Inference is very fast
    - We can combine an LM safely
  - **Cons**
    - The performance is worse compared with attention or RNN-transducer

$$p(W|O) = \sum_{Z \in f^{-1}(W)} p(Z|O)$$

$$= \sum_{Z \in f^{-1}(W)} \prod_{t=1}^{T} p(z_t | \cancel{z_{1:t-1}}, O)$$

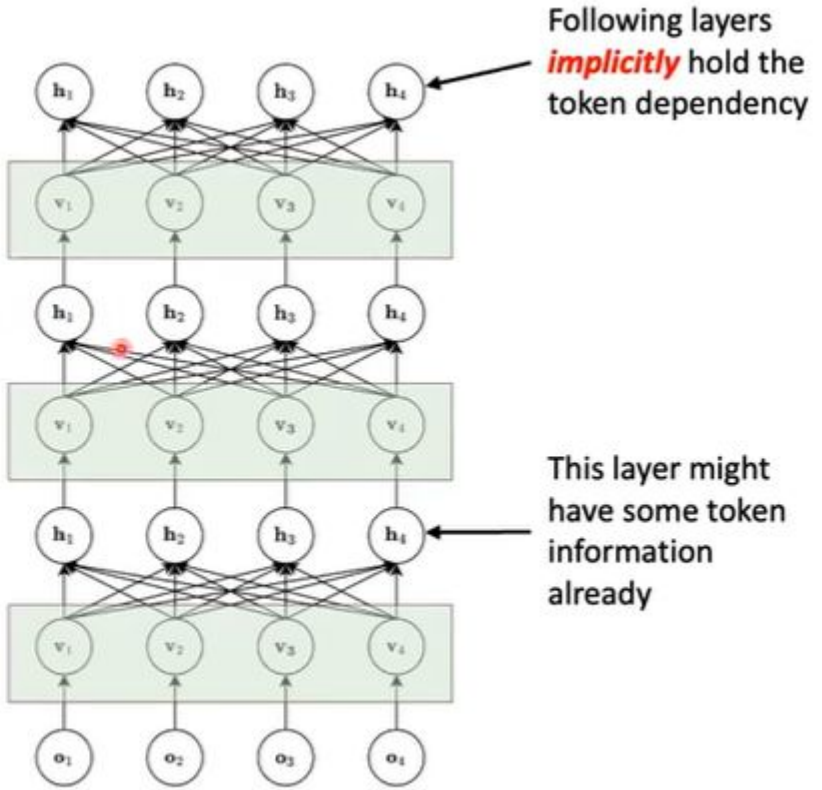
$$= \sum_{Z \in f^{-1}(W)} \prod_{t=1}^{T} p(z_t|O)$$

Implicit Conditional Information in CTC

- Does CTC really do not hold token dependency in the model?

- Deep neural network stacks the many layers
  - If the *shallow layers* start to predict the token-like information.
    e.g., $\mathbf{h}_t^l$ has some information of w_j
  - Then, the following layers may hold the token dependency ***implicitly*** thanks to the context modeling by **self-attention** or BLSTM
    $p(\mathbf{h}_t^{l+1}|\mathbf{h}_1^l, \mathbf{h}_2^l, \dots, \mathbf{h}_T^l)$

➔ CTC becomes powerful *especially* after the transformer encoder



Limitation of CTC

- Like conventional HMM-DNN models, CTC also assume **conditional independence while predicting output tokens**. In real world application usually the output tokens are dependent on each other.
- For CTC the **output (Y) length can never be greater than input (X) length**. This constraint does not affect application like ASR where mostly this constraint is satisfied.
- But for the problems like **TTS (Text to Speech) or MT (Machine Translation)** this constraint becomes a major limitation which gives rise to invention of **Encoder-Decoder architectures**.

Predicting weather condition of next 3 days

- Now suppose **Day 1 is Sunny**, **what will be Day 2?**
- We again find maximum out of these three:
  max{P(Sunny|Sunny), P(Cloudy|Sunny), P(Rainy|Sunny)}
- We find out that **P(Sunny|Sunny)** is maximum and hence **Day 2 will be Sunny** is most probable.
- Similarly, we can find out that **Day 3 being Sunny** is most probable.
- This approach is called **Greedy approach.**
- Greedy approach does not give the best possible sequence
- We can compute the probability of all possible sequences and choose the best

		Today		
		Sunny	**Cloudy**	**Rainy**
Yesterday	**Sunny**	0.5	0.375	0.125
	Cloudy	0.25	0.125	0.625
	Rainy	0.25	0.075	0.675

Predicting weather condition of next 3 days

Day 1	Day 2	Day 3	Probability of the sequence		
Sunny	Sunny	Sunny	P(Sunny|Sunny)*P(Sunny|Sunny)*P(Sunny|Sunny)	0.5*0.5*0.5	0.125
Sunny	Sunny	Cloudy	P(Sunny|Sunny)*P(Sunny|Sunny)*P(Cloudy|Sunny)	0.5*0.5*0.375	0.09375
Sunny	Sunny	Rainy	P(Sunny|Sunny)*P(Sunny|Sunny)*P(Rainy|Sunny)	0.5*0.5*0.125	0.03125
Sunny	Cloudy	Sunny	P(Sunny|Sunny)*P(Cloudy|Sunny)*P(Sunny|Cloudy)	0.5*0.375*0.25	0.046875
Sunny	Cloudy	Cloudy	P(Sunny|Sunny)*P(Cloudy|Sunny)*P(Cloudy|Cloudy)	0.5*0.375*0.125	0.0234375
Sunny	Cloudy	Rainy	P(Sunny|Sunny)*P(Cloudy|Sunny)*P(Rainy|Cloudy)	0.5*0.375*0.625	0.1171875
Sunny	Rainy	Sunny	P(Sunny|Sunny)*P(Rainy|Sunny)*P(Sunny|Rainy)	0.5*0.125*0.25	0.015625
Sunny	Rainy	Cloudy	P(Sunny|Sunny)*P(Rainy|Sunny)*P(Cloudy|Rainy)	0.5*0.125*0.075	0.0046875
Sunny	Rainy	Rainy	P(Sunny|Sunny)*P(Rainy|Sunny)*P(Rainy|Rainy)	0.5*0.125*0.675	0.0421875
Cloudy	Sunny	Sunny	P(Cloudy|Sunny)*P(Sunny|Cloudy)*P(Sunny|Sunny)	0.375*0.25*0.5	0.046875
Cloudy	Sunny	Cloudy	P(Cloudy|Sunny)*P(Sunny|Cloudy)*P(Cloudy|Sunny)	0.375*0.25*0.375	0.03515625
Cloudy	Sunny	Rainy	P(Cloudy|Sunny)*P(Sunny|Cloudy)*P(Rainy|Sunny)	0.375*0.25*0.125	0.01171875
Cloudy	Cloudy	Sunny	P(Cloudy|Sunny)*P(Cloudy|Cloudy)*P(Sunny|Cloudy)	0.375*0.125*0.25	0.01171875
Cloudy	Cloudy	Cloudy	P(Cloudy|Sunny)*P(Cloudy|Cloudy)*P(Cloudy|Cloudy)	0.375*0.125*0.125	0.00585938
Cloudy	Cloudy	Rainy	P(Cloudy|Sunny)*P(Cloudy|Cloudy)*P(Rainy|Cloudy)	0.375*0.125*0.625	0.02929688
Cloudy	Rainy	Sunny	P(Cloudy|Sunny)*P(Rainy|Cloudy)*P(Sunny|Rainy)	0.375*0.625*0.25	0.05859375
Cloudy	Rainy	Cloudy	P(Cloudy|Sunny)*P(Rainy|Cloudy)*P(Cloudy|Rainy)	0.375*0.625*0.075	0.01757813
Cloudy	Rainy	Rainy	P(Cloudy|Sunny)*P(Rainy|Cloudy)*P(Rainy|Rainy)	0.375*0.625*0.675	0.15820313
Rainy	Sunny	Sunny	P(Rainy|Sunny)*P(Sunny|Rainy)*P(Sunny|Sunny)	0.125*0.25*0.5	0.015625
Rainy	Sunny	Cloudy	P(Rainy|Sunny)*P(Sunny|Rainy)*P(Cloudy|Sunny)	0.125*0.25*0.375	0.01171875
Rainy	Sunny	Rainy	P(Rainy|Sunny)*P(Sunny|Rainy)*P(Rainy|Sunny)	0.125*0.25*0.125	0.00390625
Rainy	Cloudy	Sunny	P(Rainy|Sunny)*P(Cloudy|Rainy)*P(Sunny|Cloudy)	0.125*0.075*0.25	0.00234375
Rainy	Cloudy	Cloudy	P(Rainy|Sunny)*P(Cloudy|Rainy)*P(Cloudy|Cloudy)	0.125*0.075*0.125	0.00117188
Rainy	Cloudy	Rainy	P(Rainy|Sunny)*P(Cloudy|Rainy)*P(Rainy|Cloudy)	0.125*0.075*0.625	0.00585938
Rainy	Rainy	Sunny	P(Rainy|Sunny)*P(Rainy|Rainy)*P(Sunny|Rainy)	0.125*0.675*0.25	0.02109375
Rainy	Rainy	Cloudy	P(Rainy|Sunny)*P(Rainy|Rainy)*P(Cloudy|Rainy)	0.125*0.675*0.075	0.00632813
Rainy	Rainy	Rainy	P(Rainy|Sunny)*P(Rainy|Rainy)*P(Rainy|Rainy)	0.125*0.675*0.675	0.05695313

→ **Greedy Solution**

→ **Best Solution**

Here, we have to do 27 calculations. Choose the sequence with maximum probability. This need not give same solution as the greedy approach

CTC Inference

- How to get the result?

$$\hat{Z} = \underset{Z}{\operatorname{argmax}}\, p(Z|O)$$

- Greedy search $(\operatorname{argmax}_Z \approx (\operatorname{argmax}_{z_1}, \operatorname{argmax}_{z_2}, \ldots, \operatorname{argmax}_{z_T})$

$$\hat{Z} = \left(\underset{z_1}{\operatorname{argmax}}\, p(Z|O), \underset{z_2}{\operatorname{argmax}}\, p(Z|O), \ldots, \underset{z_T}{\operatorname{argmax}}\, p(Z|O)\right)$$

$$= \left(\underset{z_1}{\operatorname{argmax}} \prod_t p(z_t|O), \underset{z_2}{\operatorname{argmax}} \prod_t p(z_t|O), \ldots, \underset{z_T}{\operatorname{argmax}} \prod_t p(z_t|O)\right)$$

$$= \left(\underset{z_1}{\operatorname{argmax}}\, p(z_1|O), \underset{z_2}{\operatorname{argmax}}\, p(z_2|O), \ldots, \underset{z_T}{\operatorname{argmax}}\, p(z_T|O)\right)$$

$$p(Z|O) = \prod_t p(z_t|\cancel{z_{1:t-1}}, O)$$

$$= \prod_t p(z_t|O)$$

Conditional Independence assumption!

- Argmax is operated independently on t
- We can parallelize the decoding!
- CTC is also called as "**non-autoregressive** model"